



Visual Recognition using Deep Learning

Deep Neural Networks & Convolutional Neural Networks

林彥宇 教授

Yen-Yu Lin, Professor

國立陽明交通大學 資訊工程學系

Computer Science, National Yang Ming Chiao Tung University

Some slides are modified from Yung-Yu Chuang,
Sheng-Jyh Wang, Fei-Fei Li, and Justin Johnson

Outline

- Recap
- Neural networks and deep learning
- Convolutional neural networks



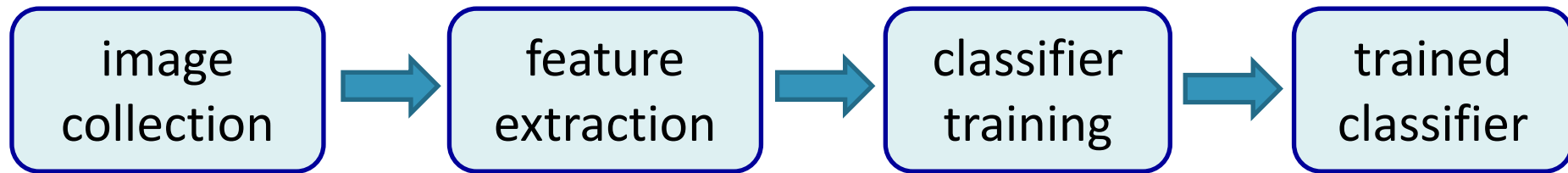
Outline

- Recap
- Neural networks and deep learning
- Convolutional neural networks

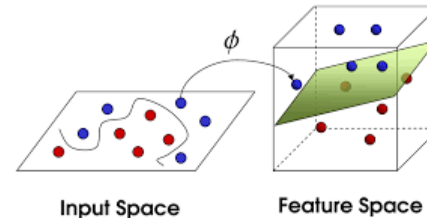


Conventional approach to object recognition

- Training phase



histogram of oriented gradients (HoG)

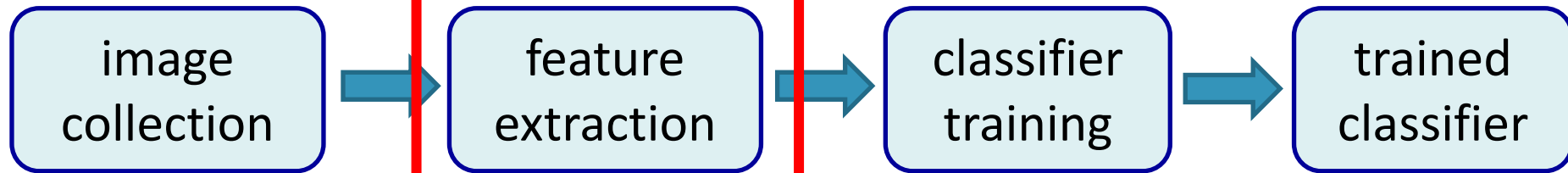


support vector machines (SVMs)

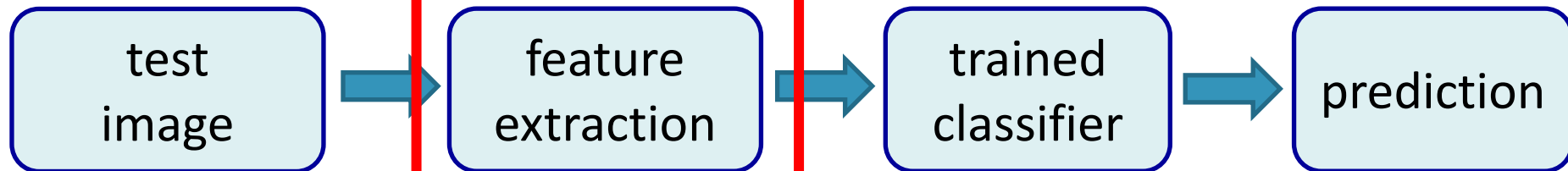
- dogs? ✗
- flowers? ○
- trains? ✗
- persons? ✗

Conventional approach to vision applications

- Training phase



- Testing phase

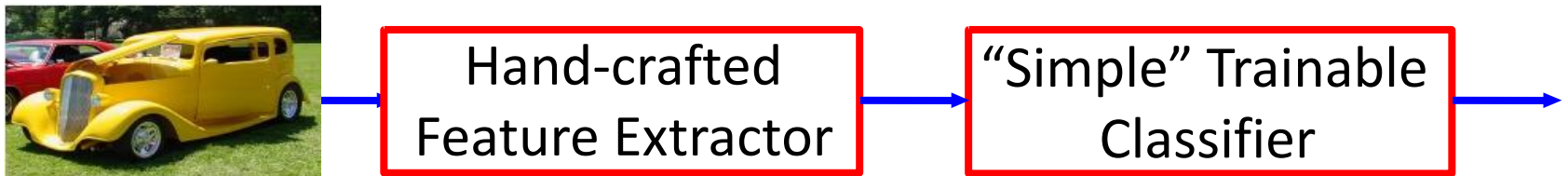


~~application~~

Conventional ML approaches vs. Deep learning

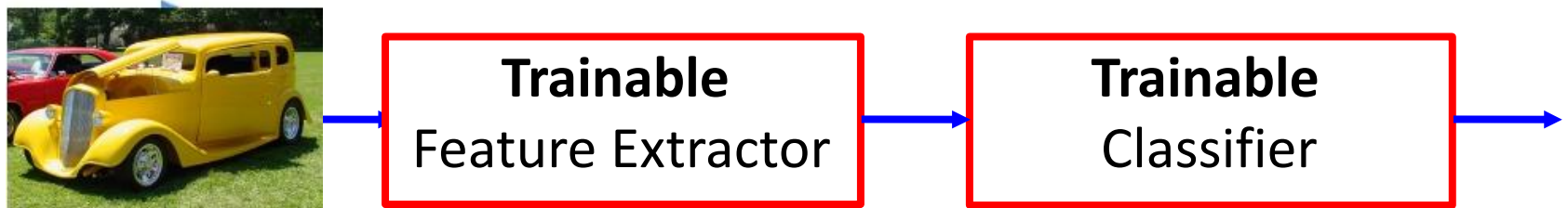
- Conventional approaches

- **Fixed/engineered** features + trainable classifier



- Deep learning / End-to-end learning / Feature learning

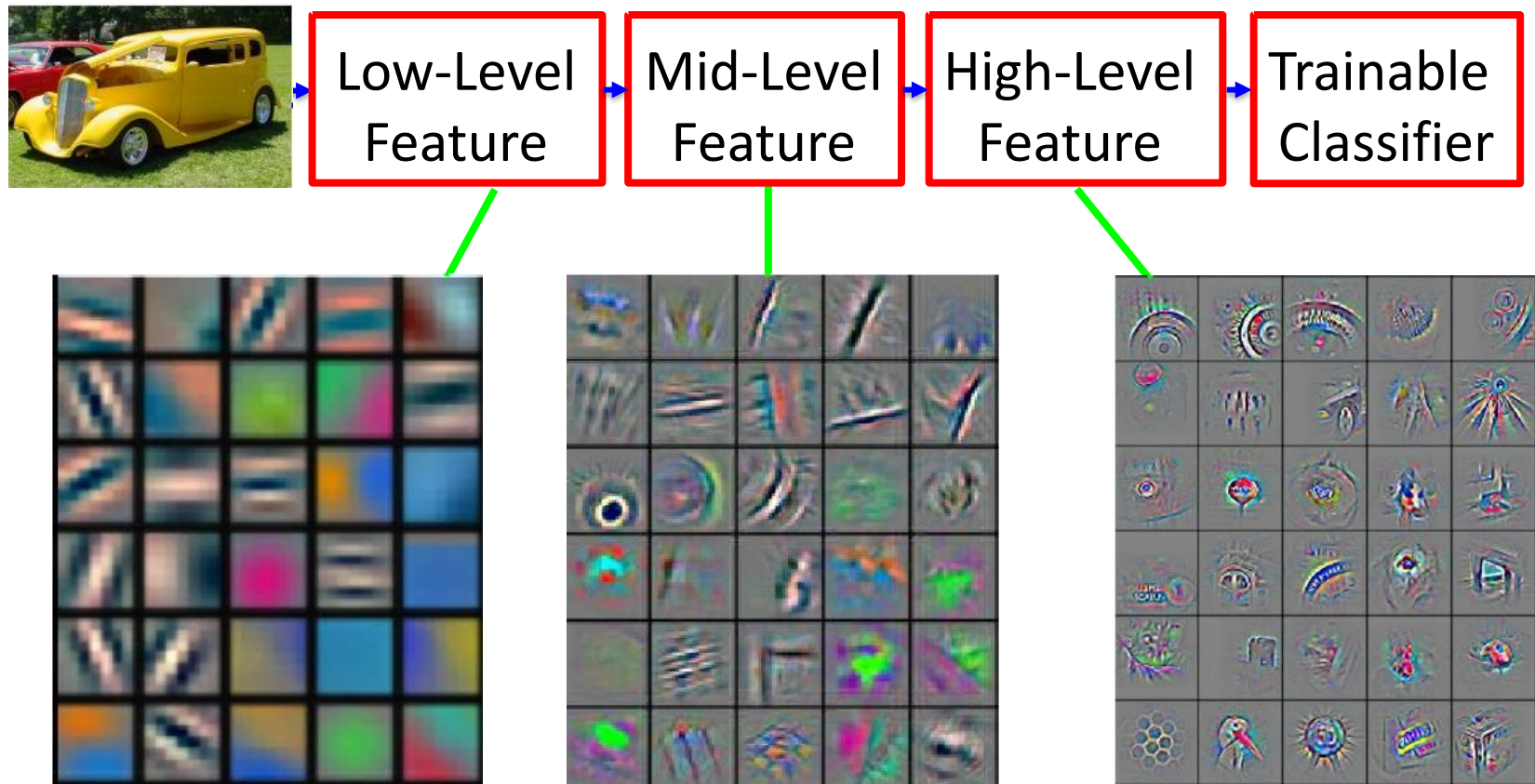
- **Trainable** features + trainable classifier



slide: Y LeCun & MA Ranzato



Deep learning = Learning hierarchical representations



slide: Y LeCun & MA Ranzato



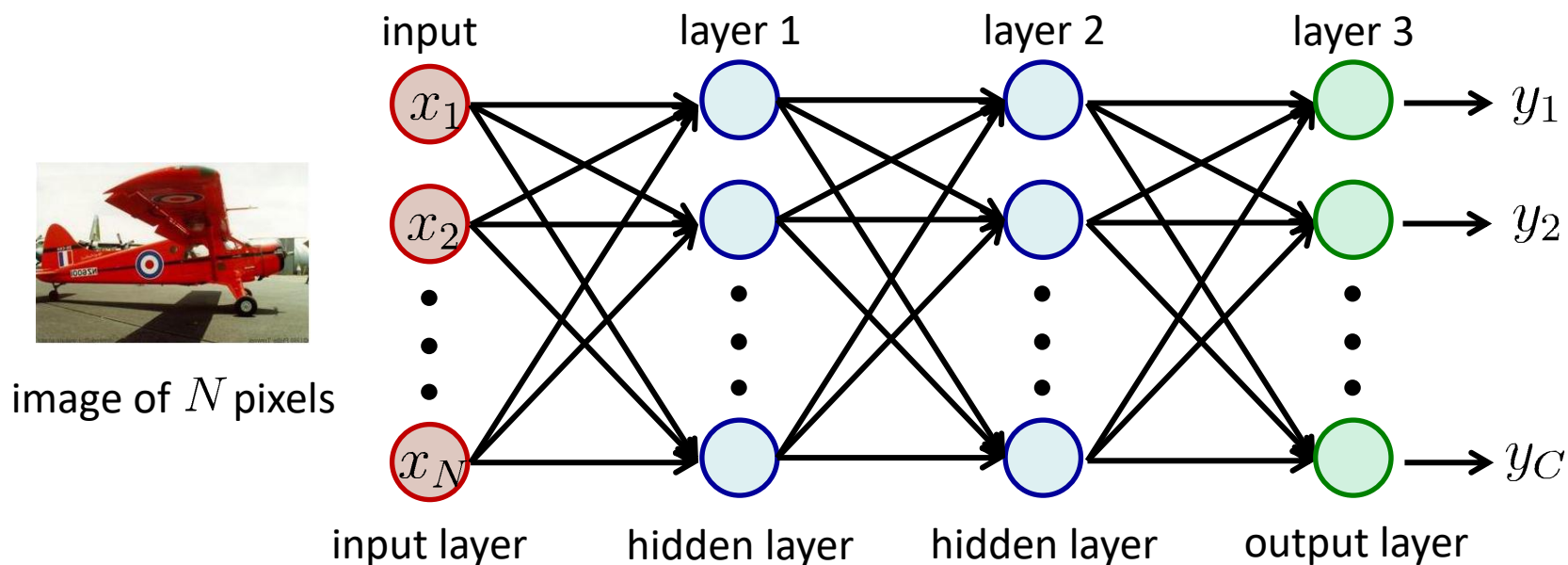
Outline

- Recap
- Neural networks and deep learning
- Convolutional neural networks



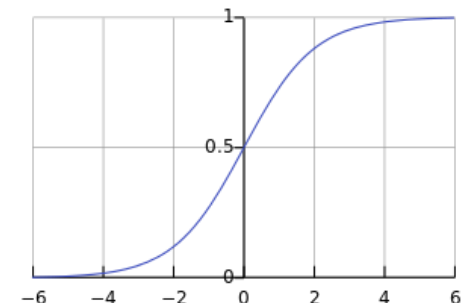
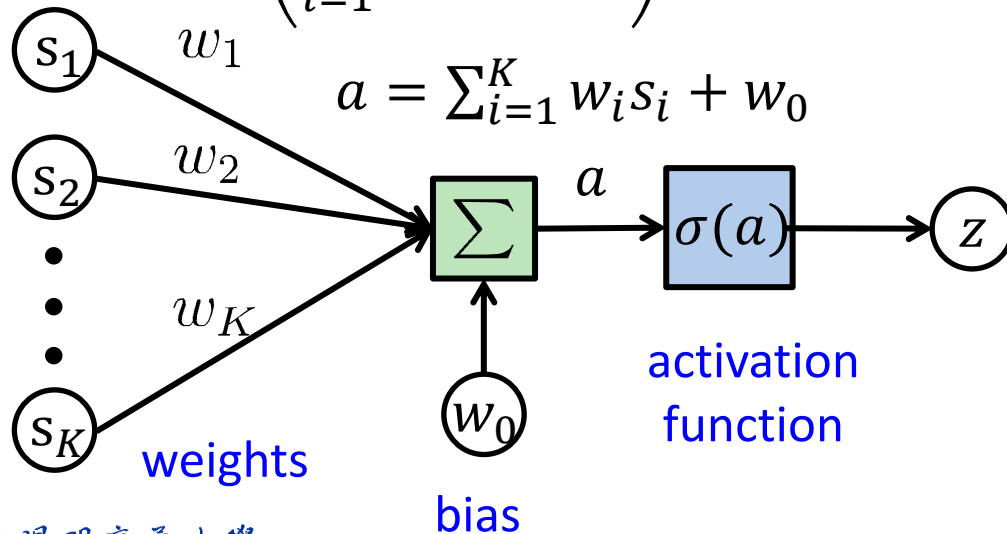
Neural networks and neurons

- Neural networks are presented as layers of interconnected neurons
 - Each layer of neurons takes messages from output of previous layer



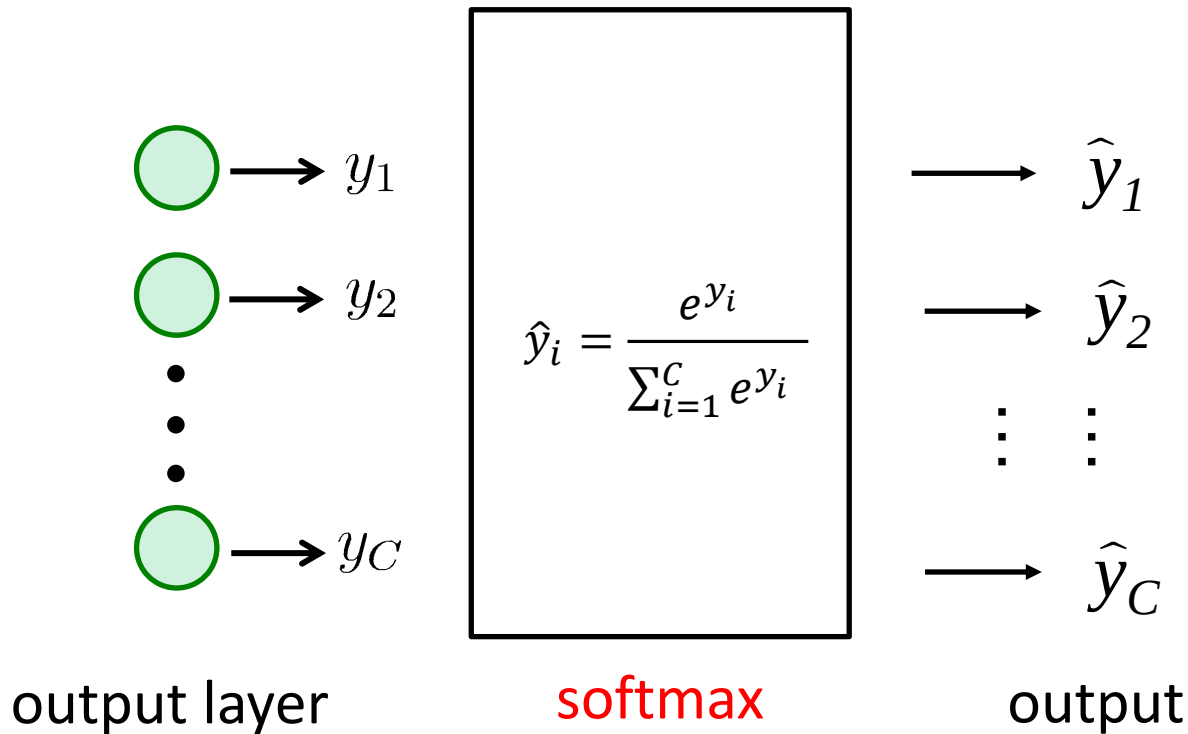
A neuron

- A function $f : R^K \mapsto R$
 - Map K inputs to 1 output
 - Compute the biased weighted sum
 - Apply a non-linear mapping function (activation function)
 - $f(\mathbf{s}) = \sigma \left(\sum_{i=1}^K w_i s_i + w_0 \right)$, where $\sigma(a) = \frac{1}{1 + \exp(-a)}$



sigmoid function

Softmax for multi-class classification



- A probability distributions
- Larger output leads to higher probability
- All positive
- Sum to 1

Training neural networks

- Collect a set of labeled training data $D = \{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N$
- Training neural networks: Finding network parameters $\mathbf{w}$ to minimize the loss between true training label $\mathbf{y}_i$ and the estimated label, e.g.,

$$E(\mathbf{w}) = \sum_{i=1}^N \|\mathbf{y}_i - g_{\mathbf{w}}(\mathbf{x}_i)\|^2$$

- Minimization can be done by **gradient descent** if $E(\mathbf{w})$ is differentiable with respect to $\mathbf{w}$
- **Backpropagation**: a widely used method for optimizing multi-layer neural networks



Gradient descent

- The simplest approach is to update optimization variable set $\mathbf{w}$ by a displacement in the negative gradient direction

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E(\mathbf{w}^{(\tau)})$$

- This is a **steepest descent** algorithm
- $\eta > 0$ is the **learning rate**
- This is a batch method, as evaluation of ∇E involves the entire data set
- A range of starting points $\{\mathbf{w}^{(0)}\}$ may be needed, in order to find a satisfactory minimum



Stochastic gradient descent

- Stochastic gradient descent has proved useful in practice when training neural networks on a large data set
- The error function needs to comprise a sum of terms, one for each data point, i.e.,

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w})$$

- Sum-of-squares error for regression

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2$$

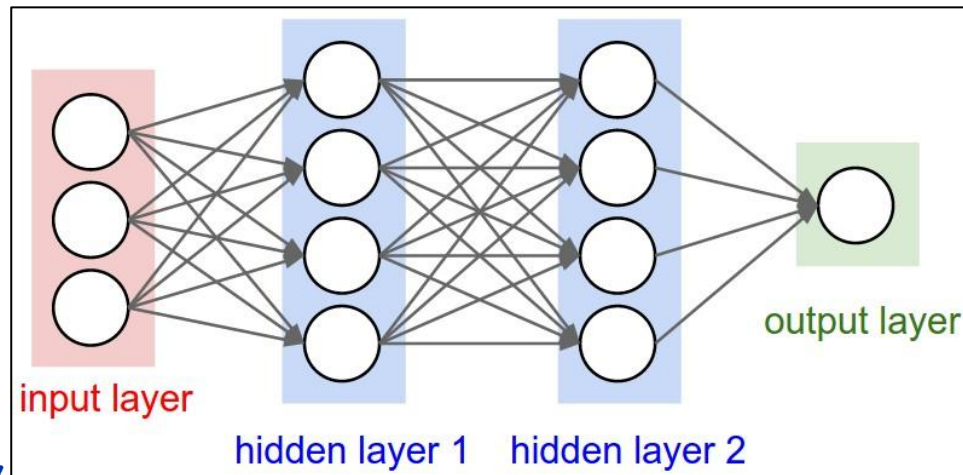
- Cross-entropy error for classification

$$E(\mathbf{w}) = - \sum_{n=1}^N \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\}$$



Optimization

- **Mini-batch SGD** (stochastic gradient descent)
- Loop:
 1. **Sample** a batch of data
 2. **Forward** prop it through the network and get loss
 3. **Backprop** to calculate the gradients
 4. **Update** the parameters using the gradient



Error backpropagation

- The computational cost of gradient descent mainly lies in the evaluation of gradient at each iteration
- In **feed-forward neural networks**, the gradient of an error function $E(\mathbf{w})$ can be efficiently evaluated via an algorithm called **error backpropagation**



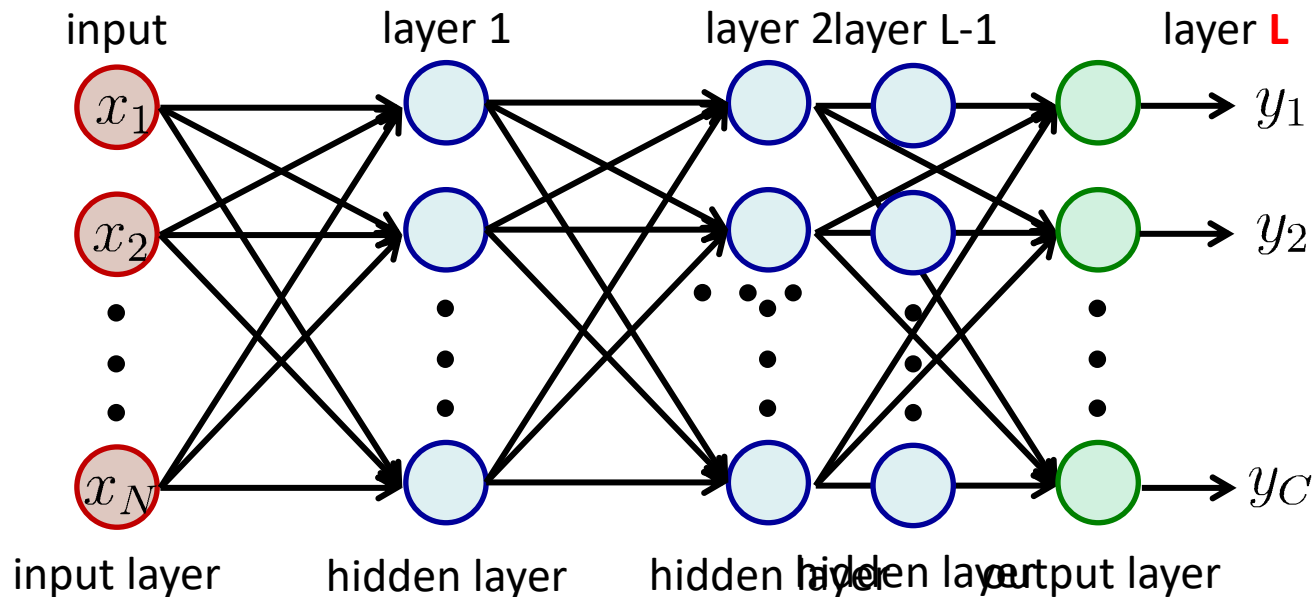
Details of neural networks

- Appendix covers
 - Neural networks
 - Gradient descent and its variants
 - Error backpropagation algorithm



What is deep neural networks (DNN)

- DNN is neural networks with many hidden layers

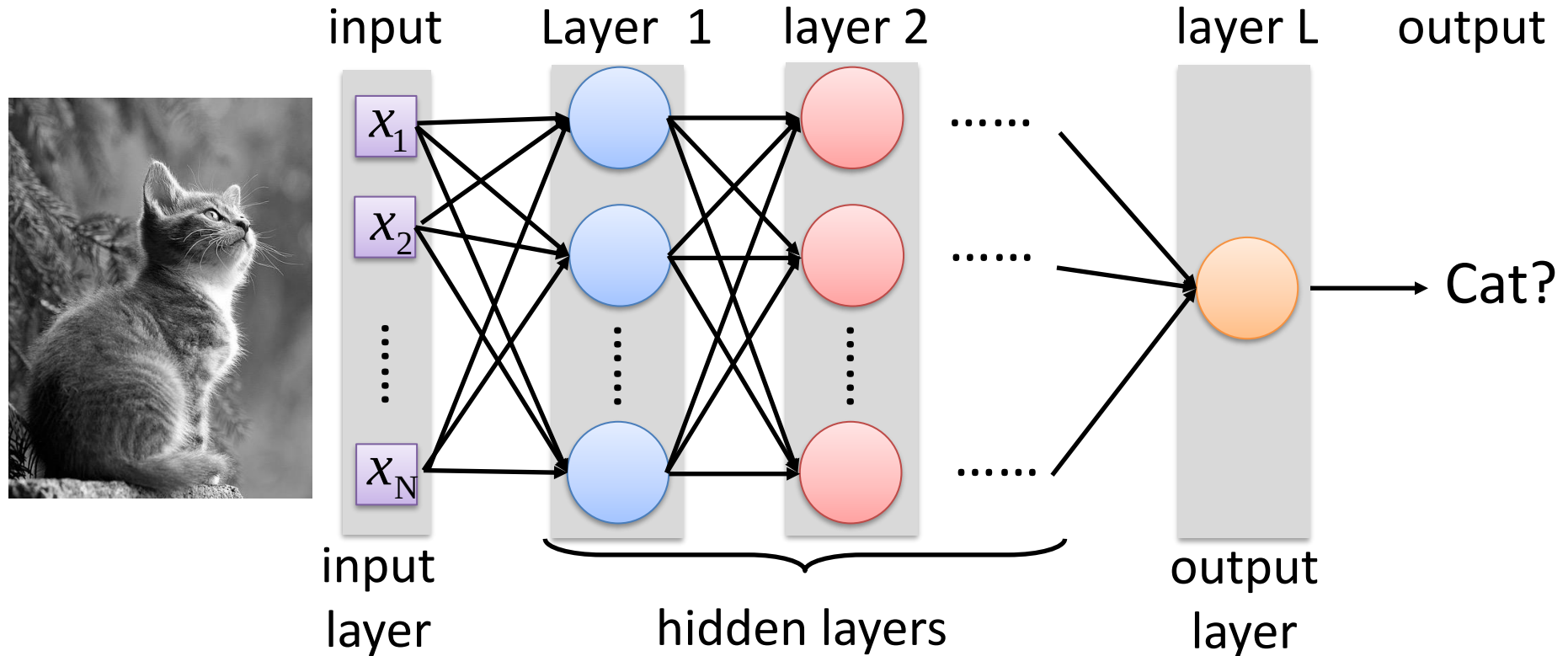


Outline

- Recap
- Neural networks and deep learning
- Convolutional neural networks

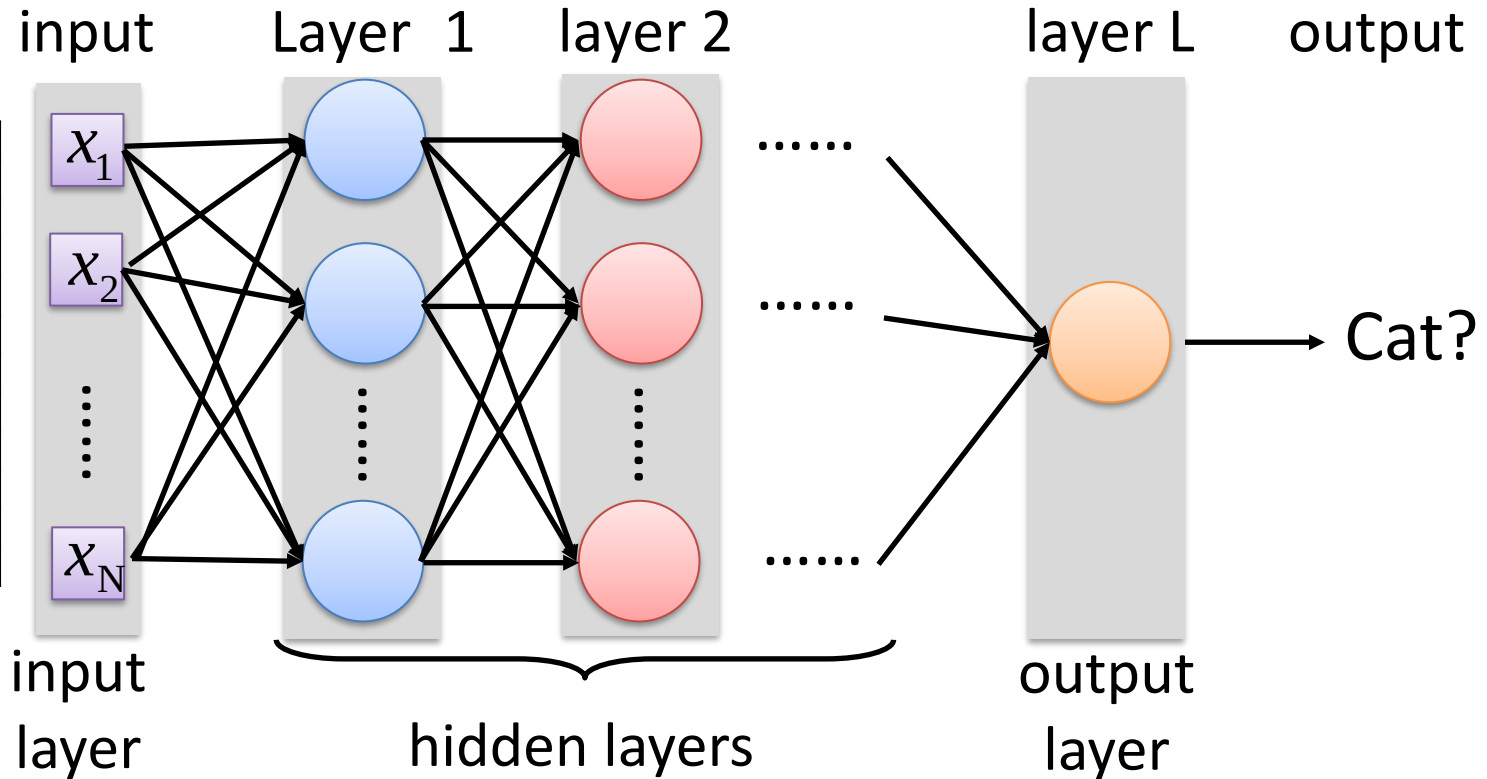


Deep learning for object recognition



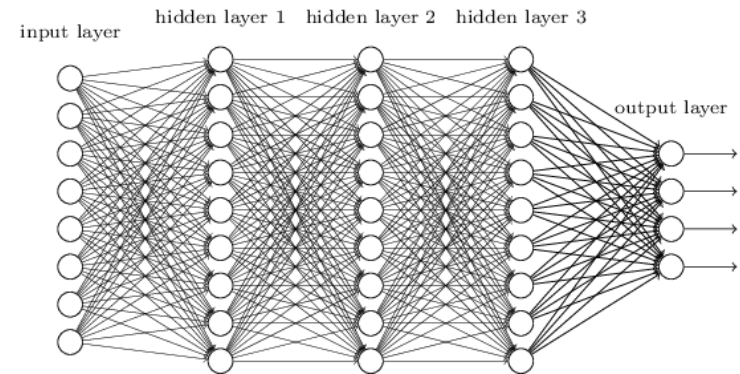
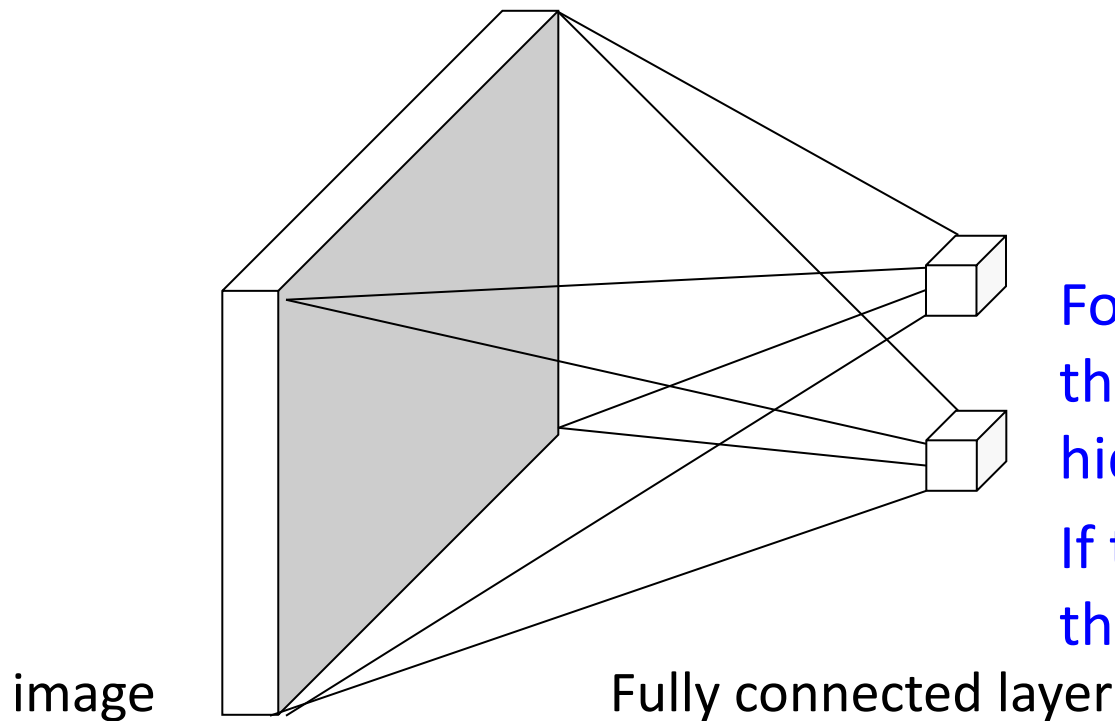
Deep learning for object recognition

243	239	240	225	206	185	188
242	239	218	110	67	31	34
243	242	123	58	94	82	132
235	217	115	212	243	236	247
233	208	131	222	219	226	196
232	217	131	116	77	150	69
232	232	182	186	184	179	159
232	236	201	154	216	133	129
235	238	230	128	172	138	65
237	236	247	143	59	78	10
234	237	245	193	55	33	115
248	245	161	128	149	109	138



Deep learning for object recognition

- Too many links when applying fully connected layers to images



For a 1000x1000 image, there are 1M links for a hidden node.

If there are 1M neurons, there are 10^{12} parameters.

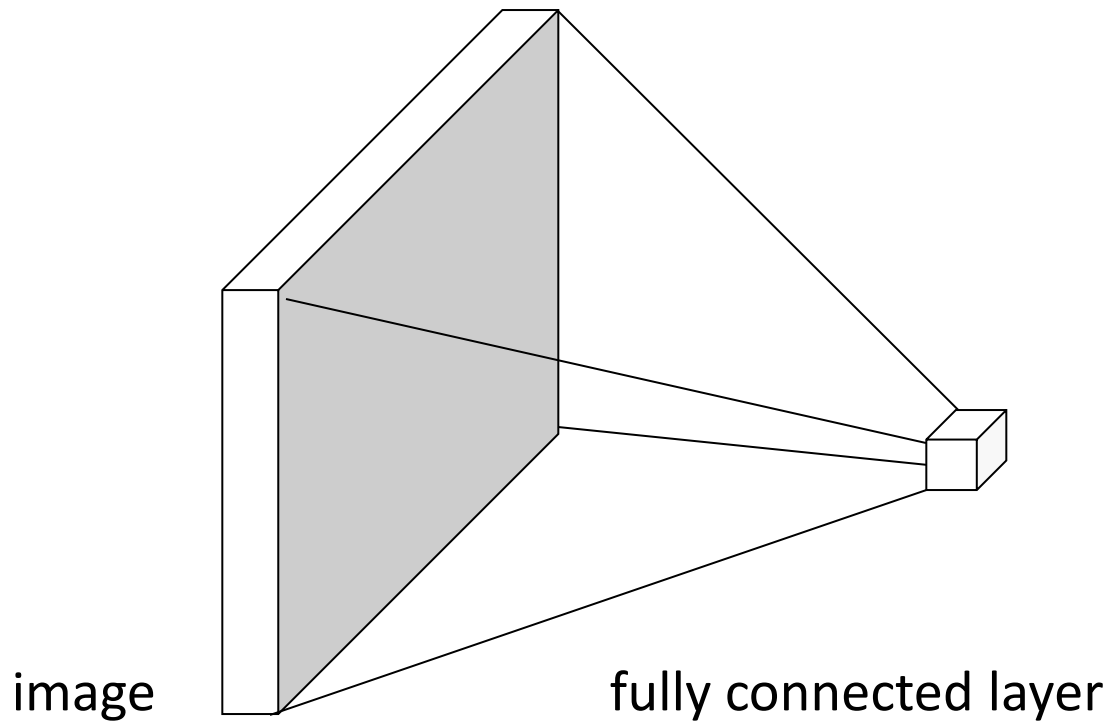
Convolutional neural networks (CNN)

- CNN: a multi-layer neural network with
 1. Local connectivity
 2. Weight sharing
- Why local connectivity?
 - Spatial correlation is local (locality of spatial dependencies)
 - Reduce # of parameters



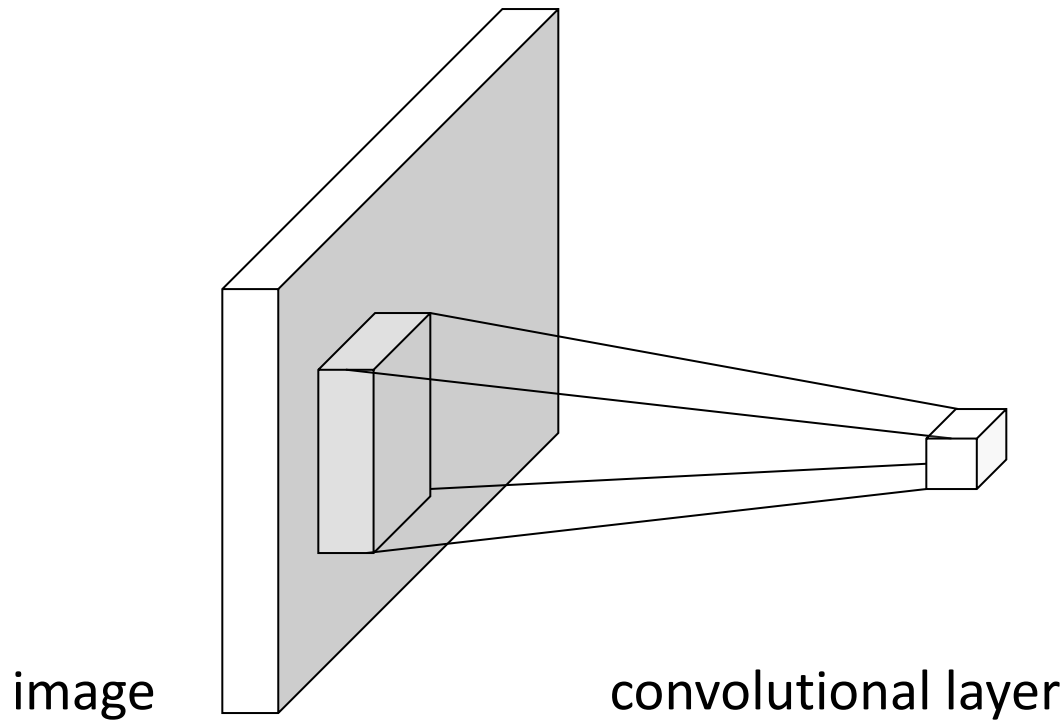
Local connectivity

- Each neuron in the convolutional layer is connected a local region, instead of the whole input image



Local connectivity

- Each neuron in the convolutional layer is connected a local region, instead of the whole input image



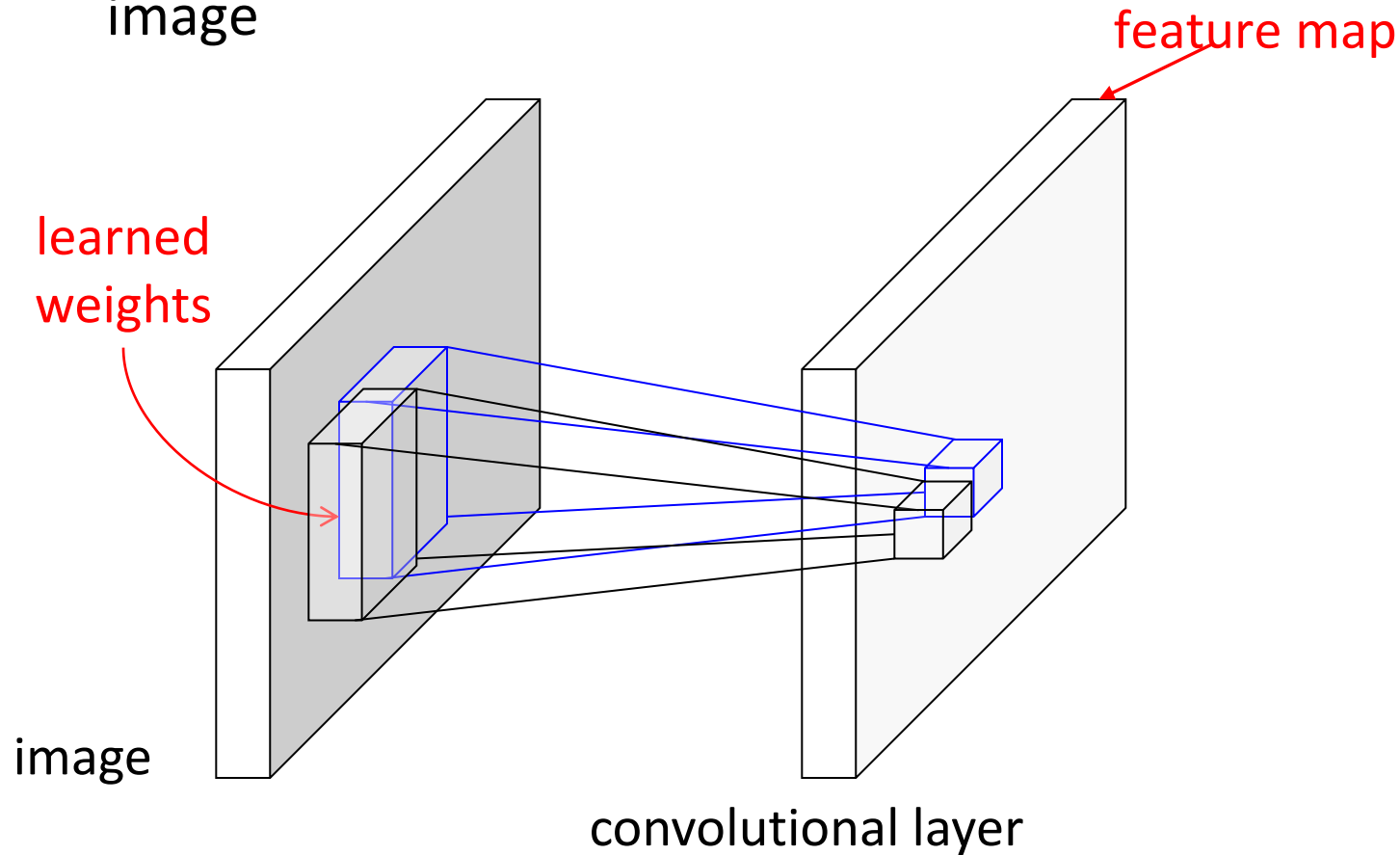
Convolutional neural networks (CNN)

- CNN: a multi-layer neural network with
 1. Local connectivity
 2. Weight sharing
- Why local connectivity?
 - Spatial correlation is local (locality of spatial dependencies)
 - Reduce # of parameters
- Why weight sharing?
 - Same statistics is at different locations (stationarity of statistics)
 - Reduce # of parameters

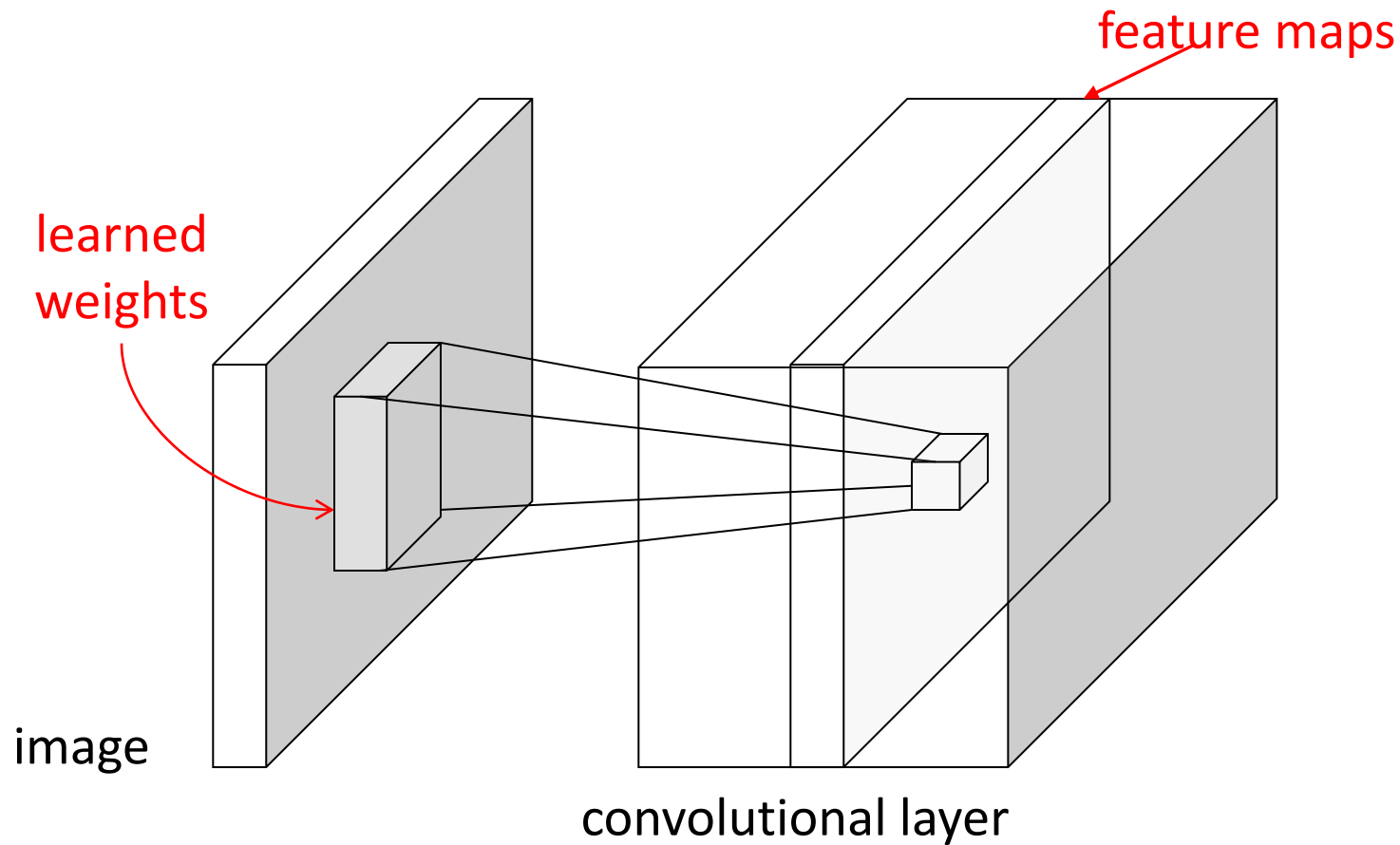


Weight sharing

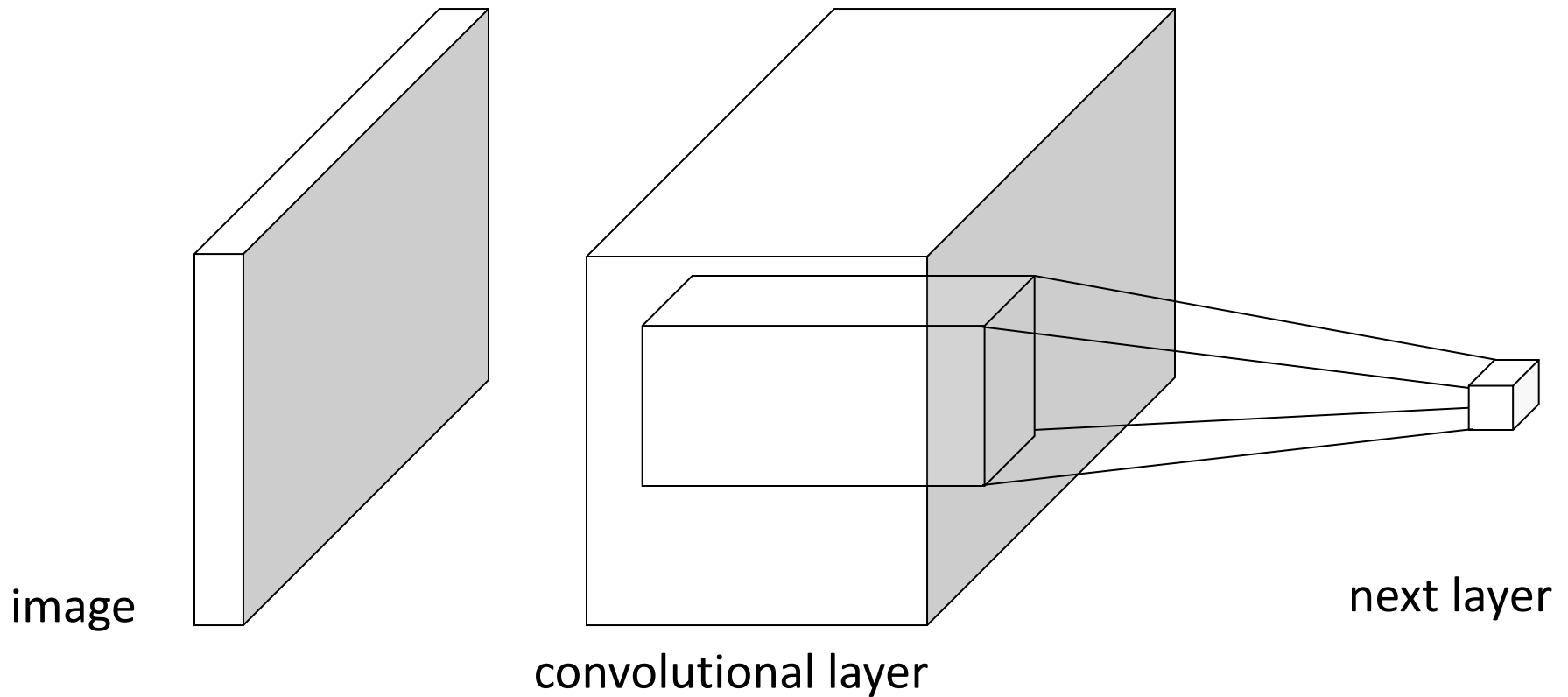
- We compute the same statistics at different locations of an image



Multiple feature maps by learning multiple filters



Stacking convolutional layers



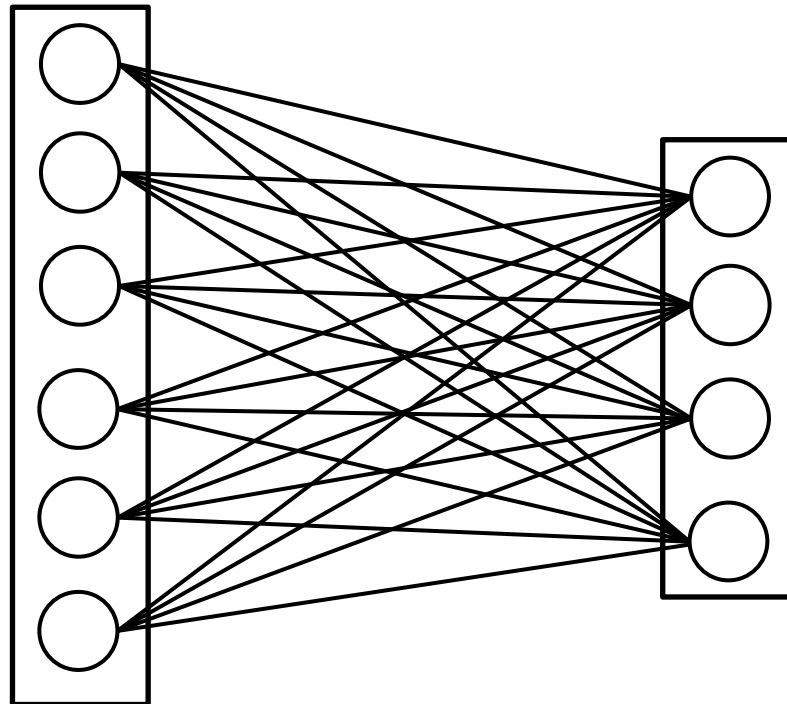
Building blocks of CNN

- Convolutional layer (CONV)
- Pooling layer (POOL)
- Fully connected layer (FC)

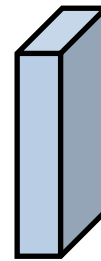
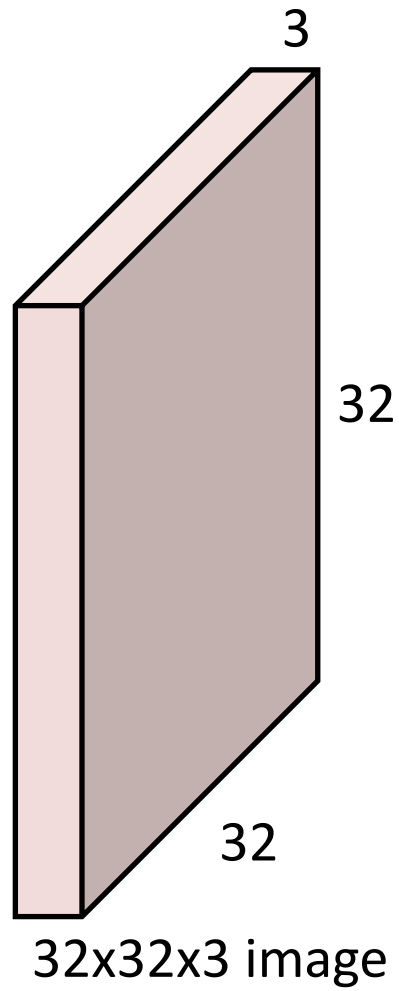


Fully connected layer

- A neuron is fully connected to all neurons in the previous layer
- Perform biased weighted sum followed by a non-linear mapping
- Deliver the output to all neurons in the next layer

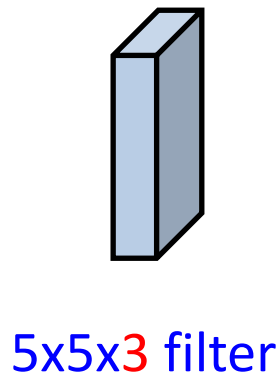
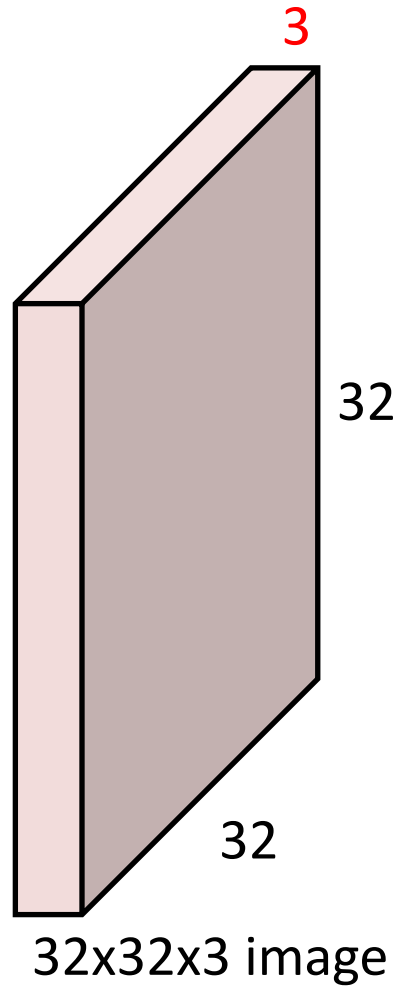


Convolutional layer



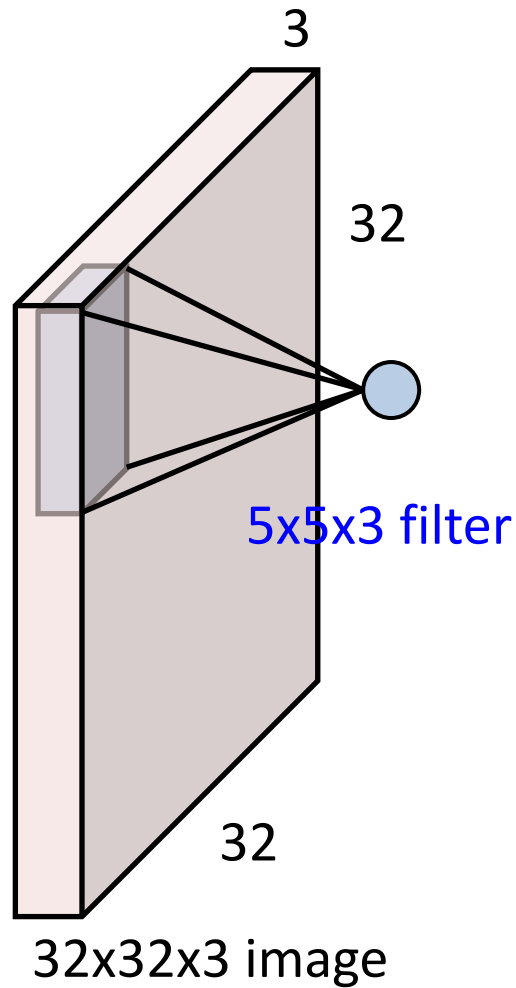
5x5x3 filter

Convolutional layer

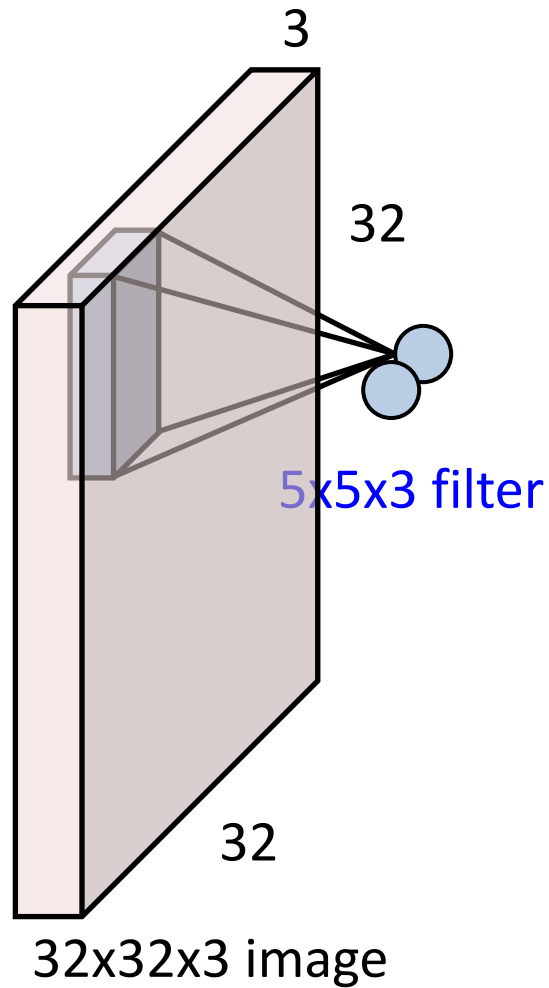


- The dimension of a filter is 3
- The third dimension is the same as the number of channels of the input image

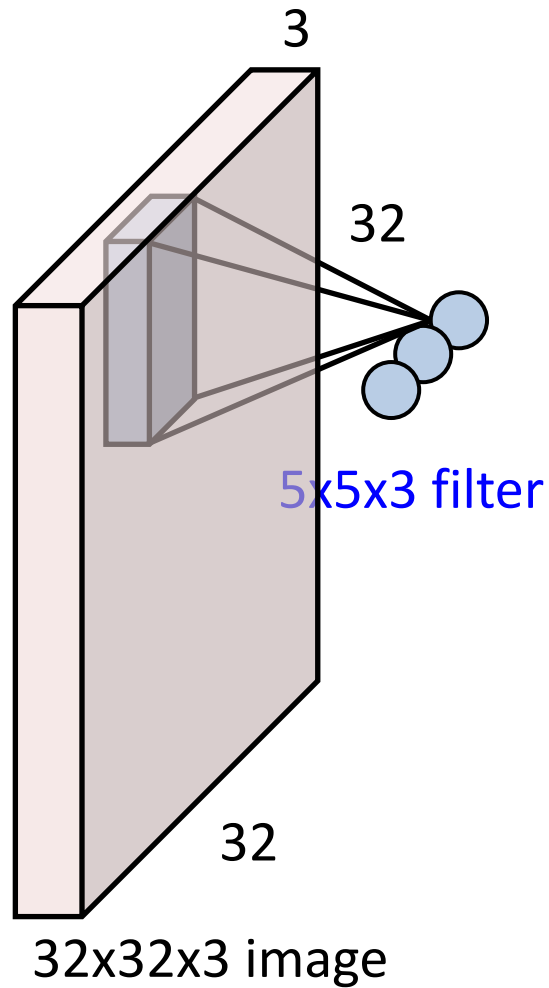
Convolutional layer: filtering + non-linear mapping



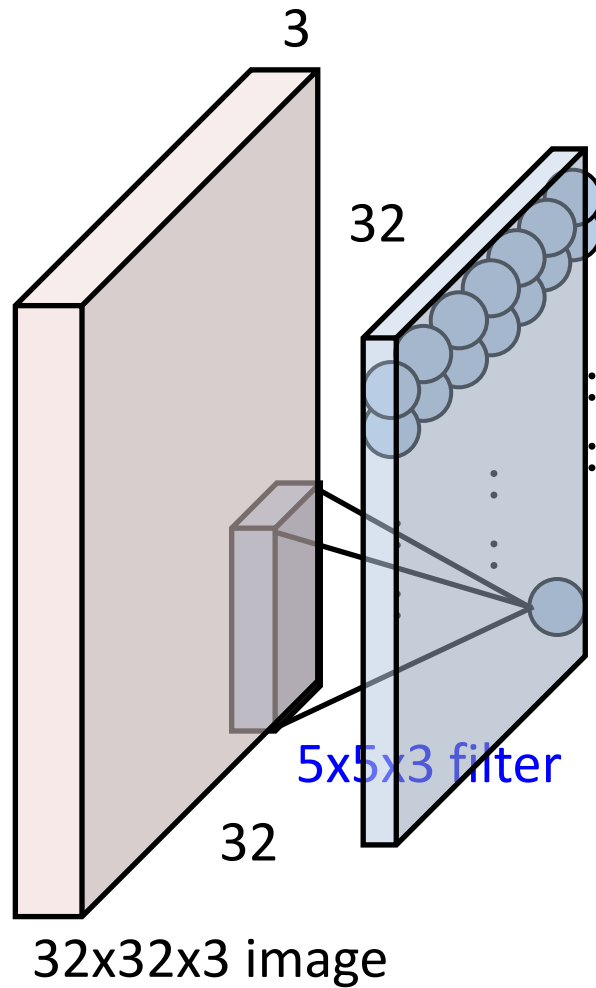
Convolutional layer: filtering + non-linear mapping



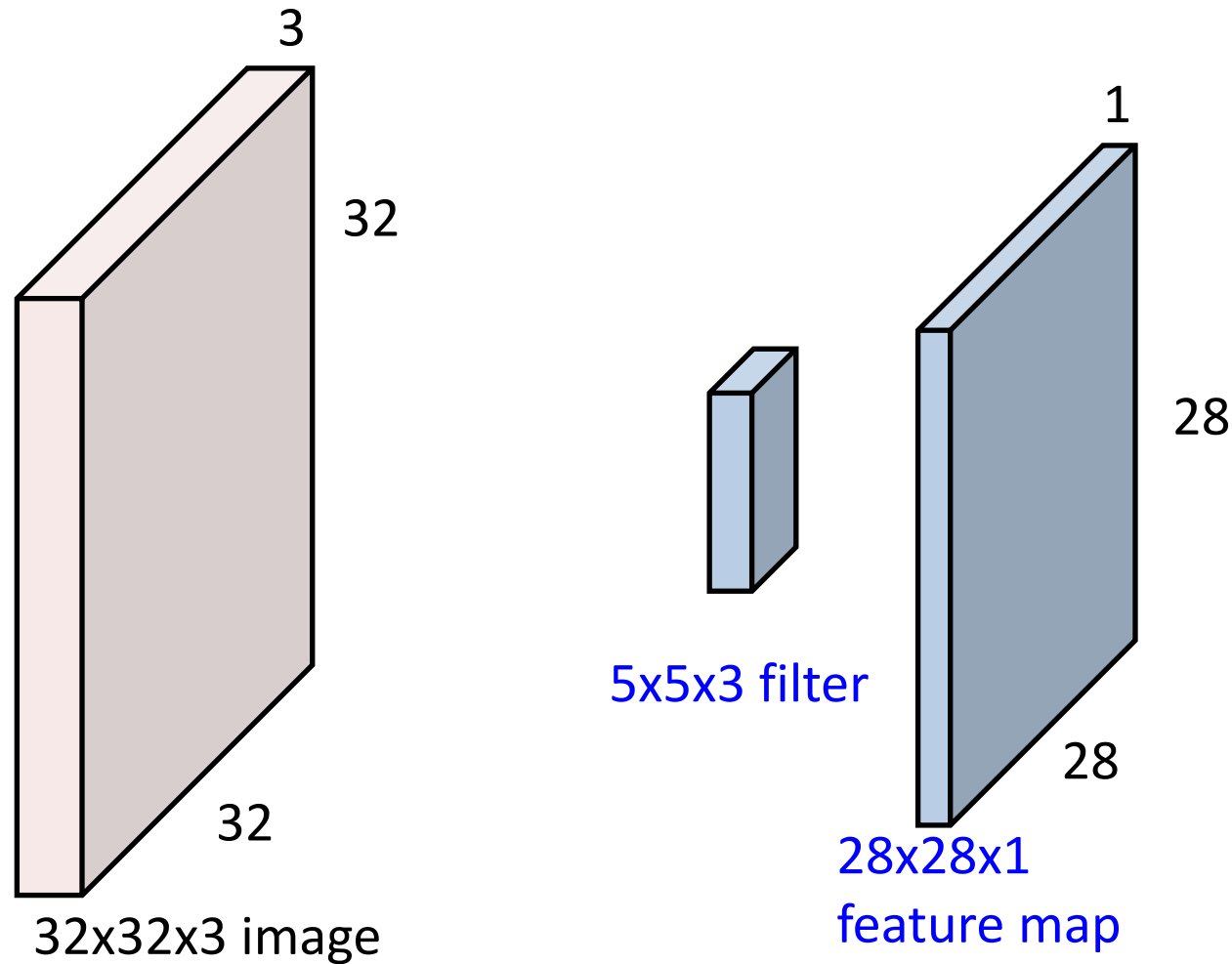
Convolutional layer: filtering + non-linear mapping



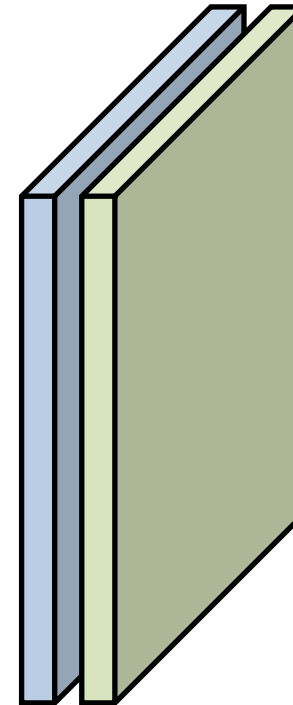
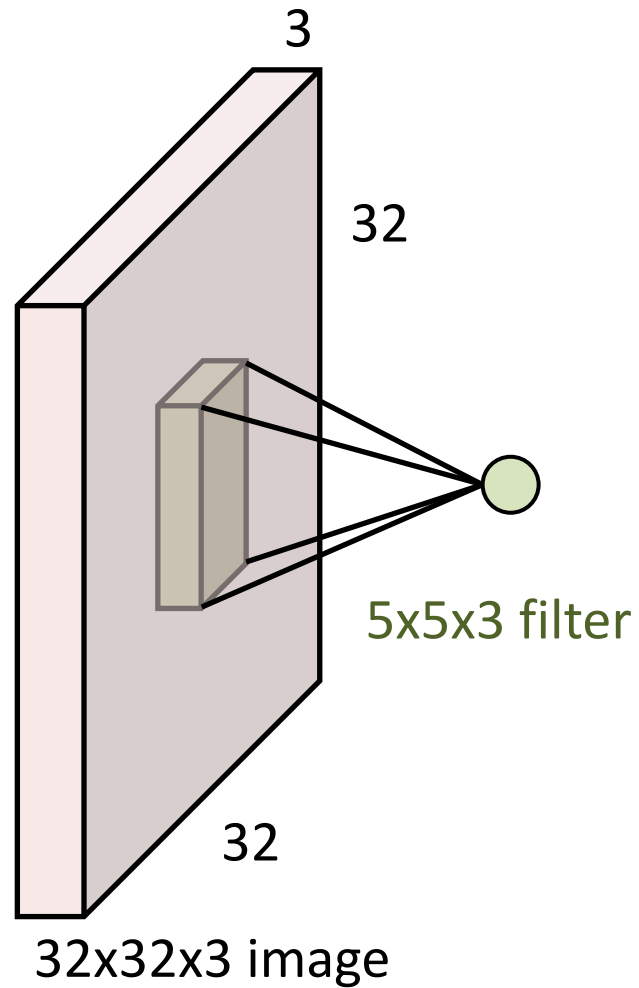
Convolutional layer: filtering + non-linear mapping



Convolutional layer: A filter yields a feature map

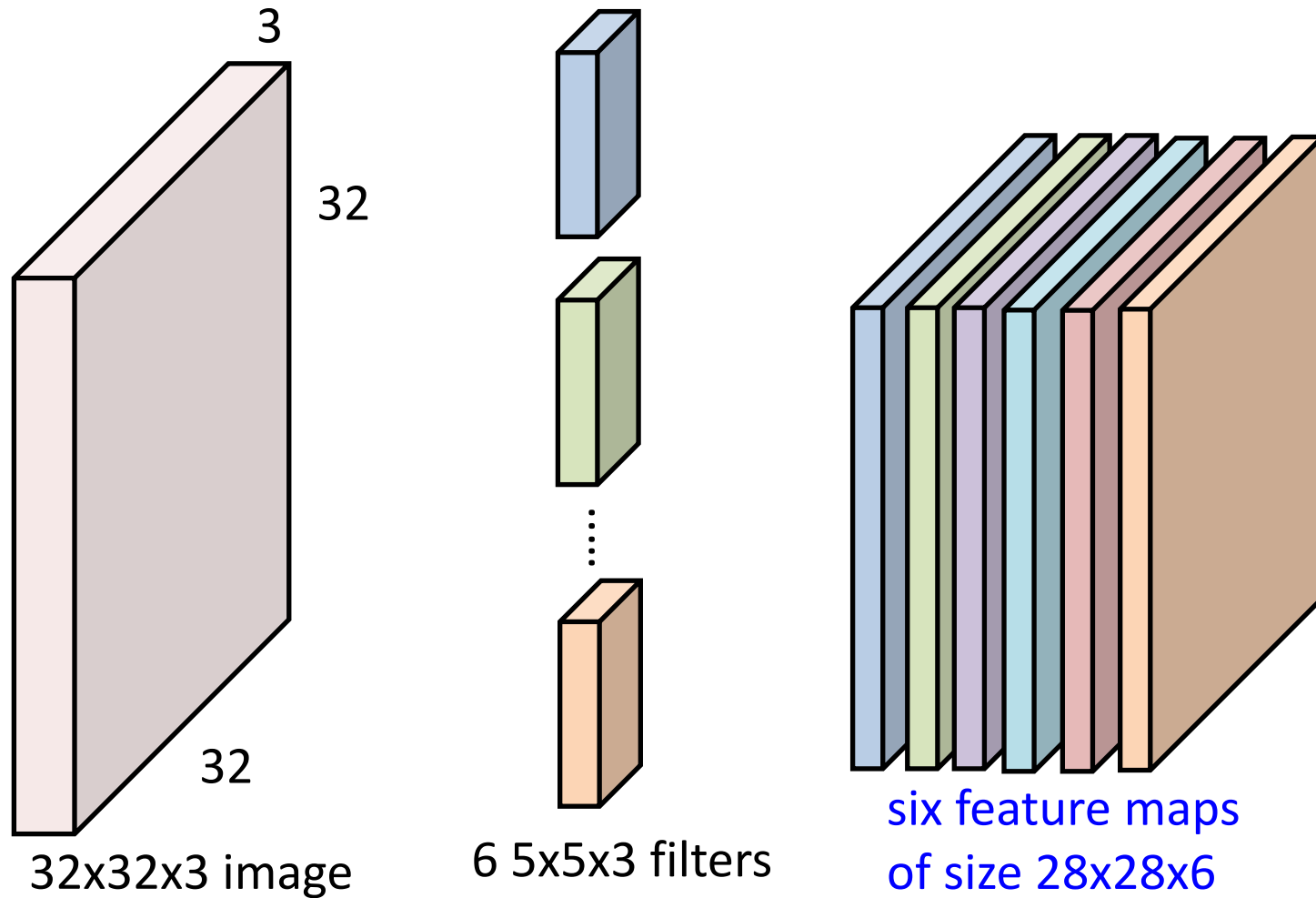


Convolutional layer: The second filter



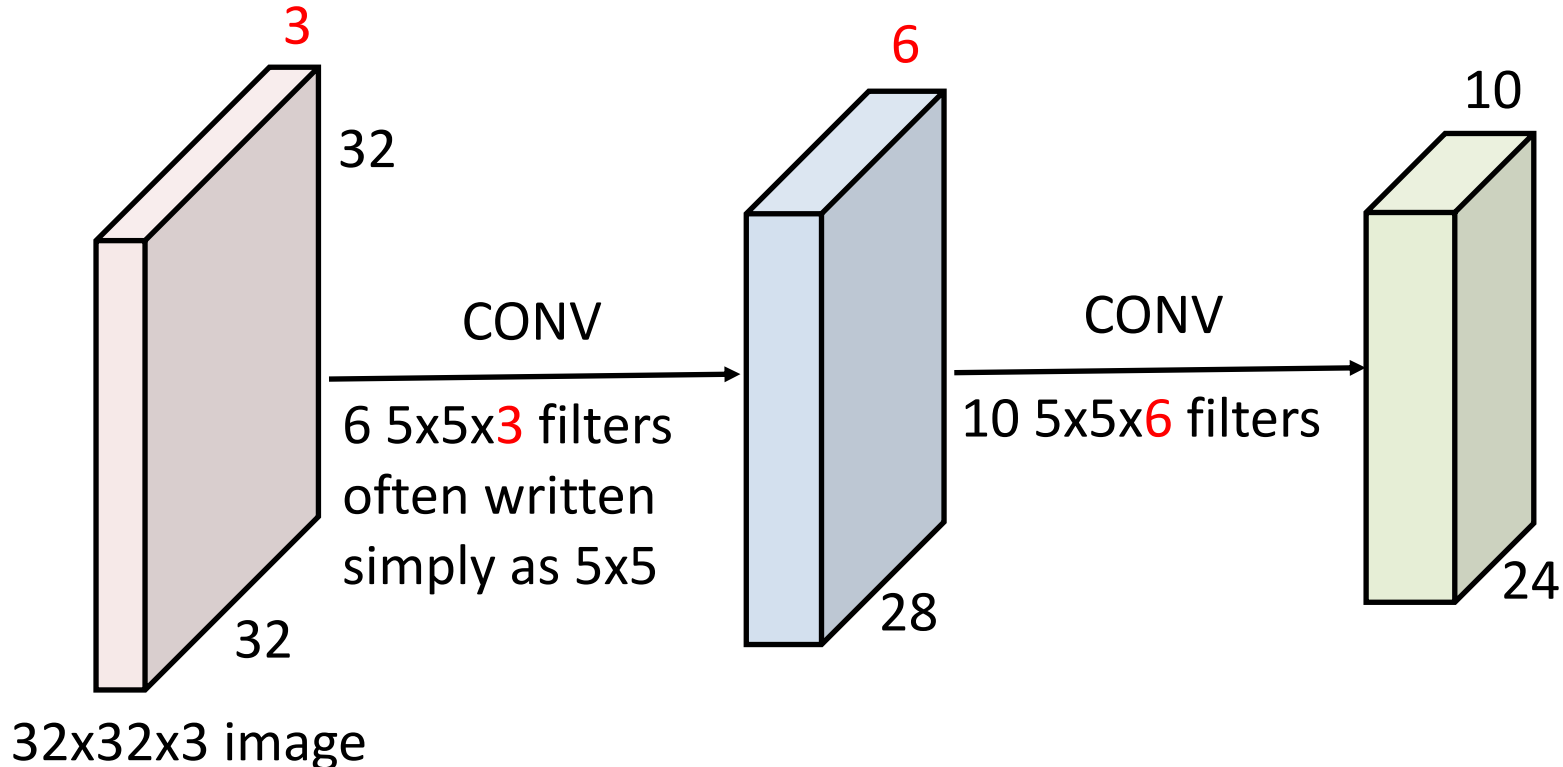
two feature maps
of size 28x28x2

Convolutional layer: N filters yield N feature maps



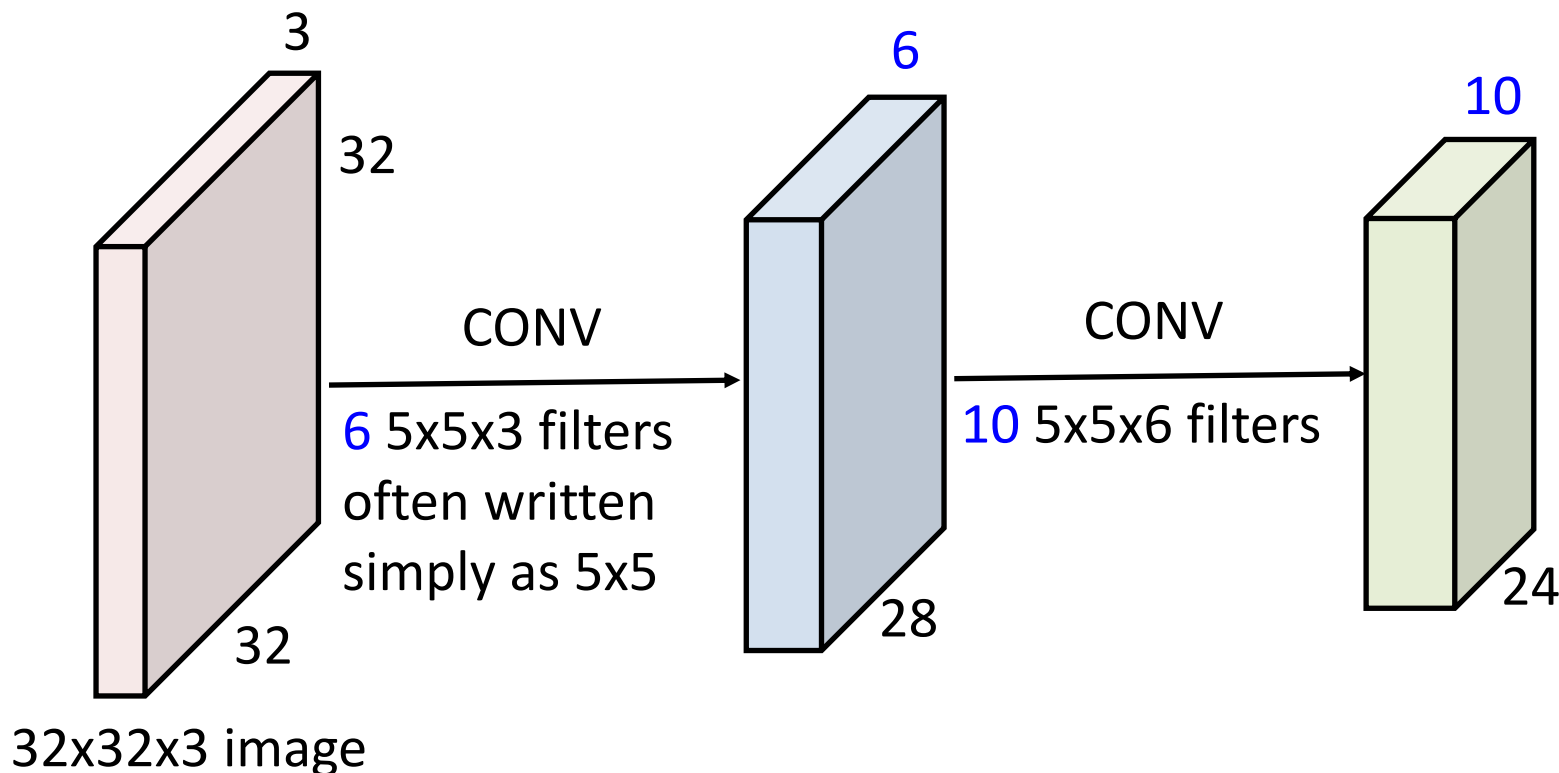
Convolutional layers

- A convolution (CONV) layer performs convolution followed by an activation function
- CNN often consists of several CONV layers.



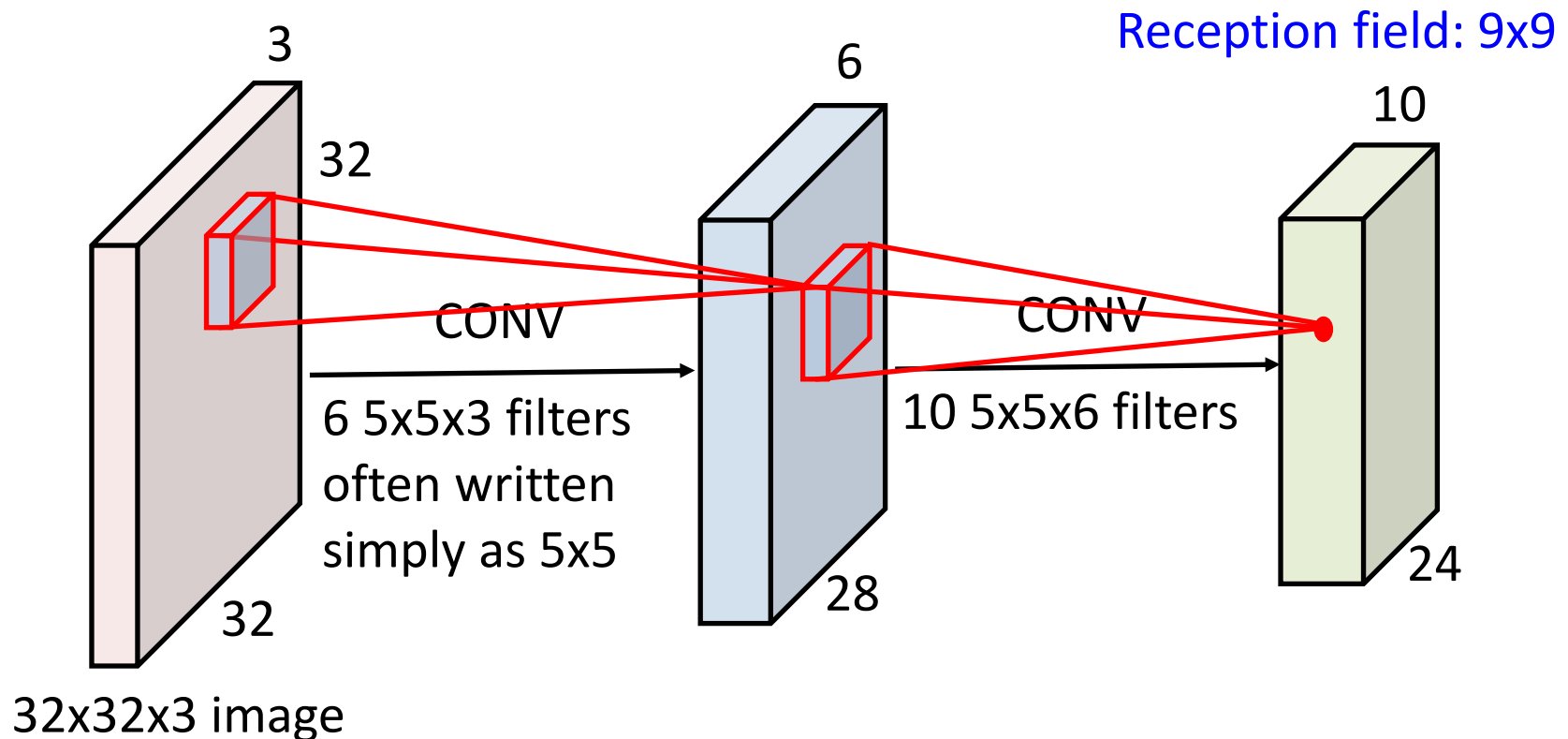
Convolutional layers

- A convolution (CONV) layer performs convolution followed by an activation function
- CNN often consists of several CONV layers.

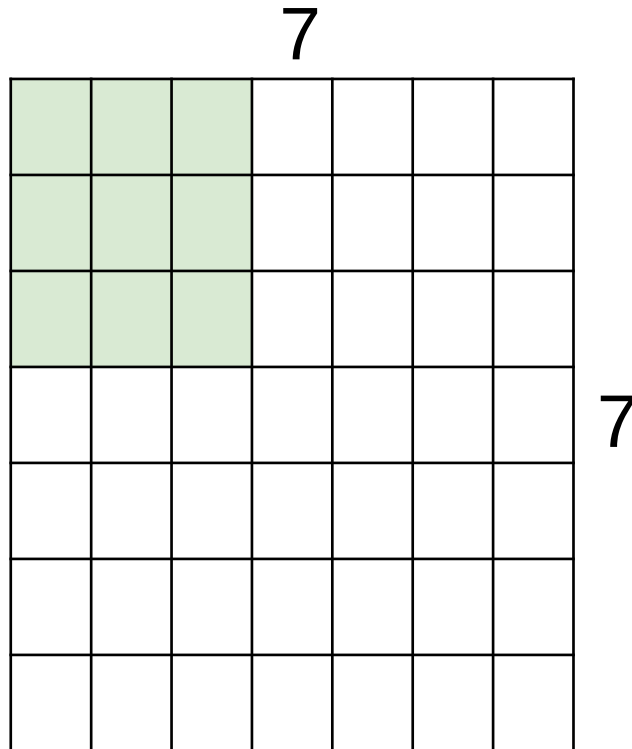


Convolutional layer: Receptive field

- How large is the area influencing a pixel of the activation map on the input image

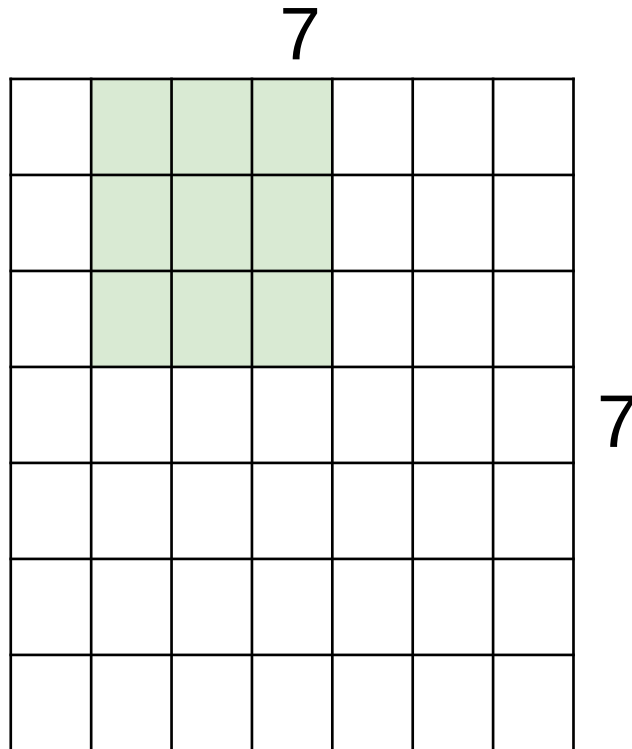


Convolutional layer: Size of a feature map



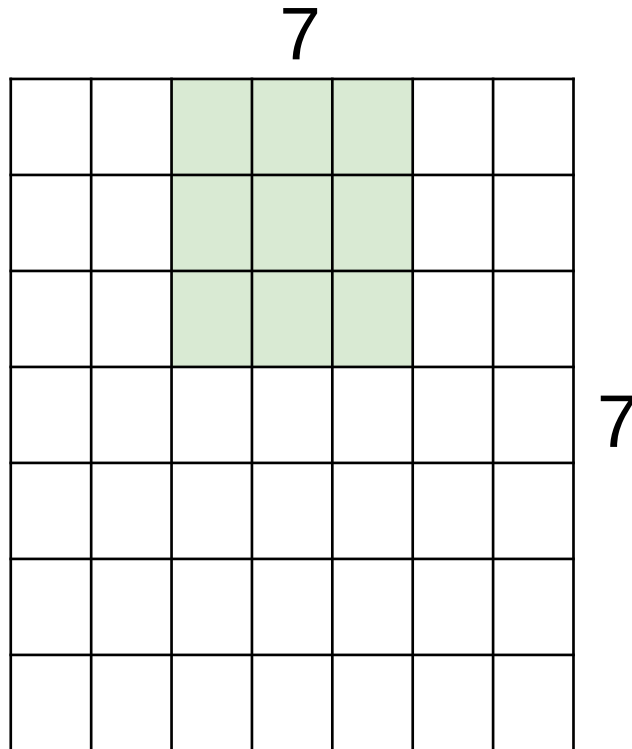
- Assume the input is 7x7 and the filter is 3x3, what is the size of the output?

Convolutional layer: Size of a feature map



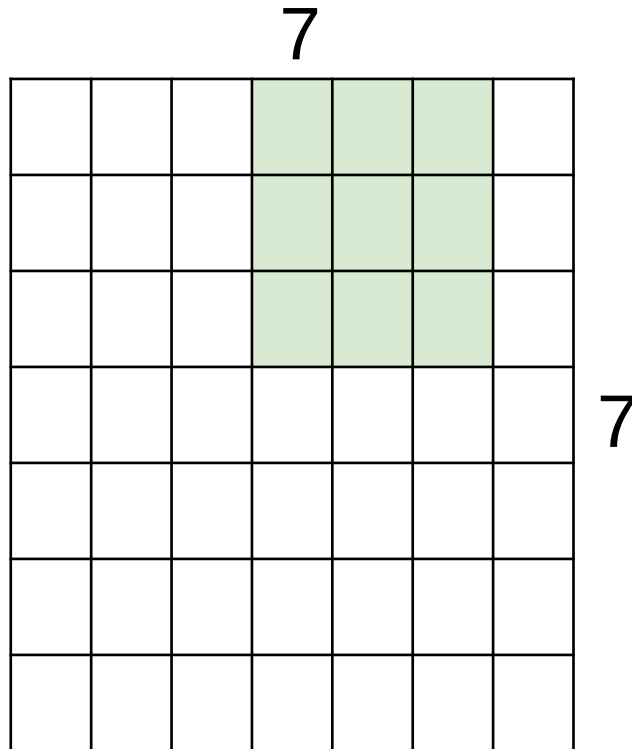
- Assume the input is 7x7 and the filter is 3x3, what is the size of the output?

Convolutional layer: Size of a feature map



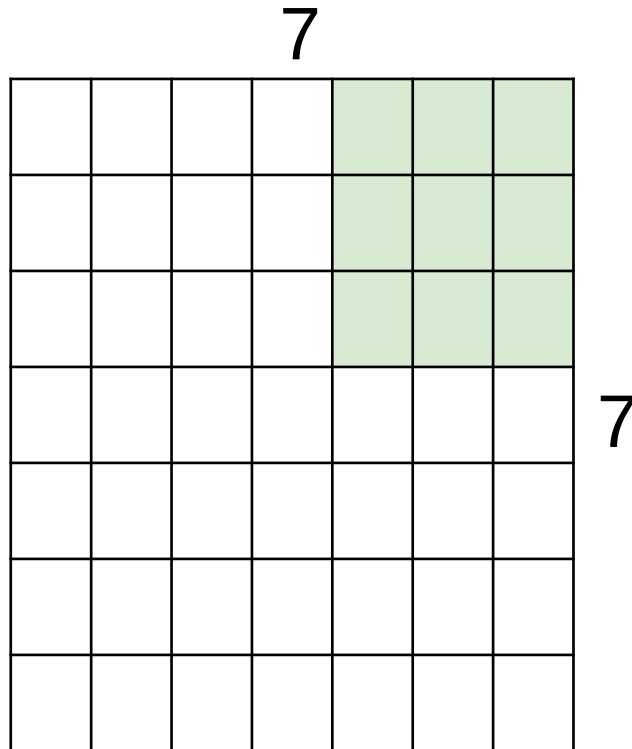
- Assume the input is 7x7 and the filter is 3x3, what is the size of the output?

Convolutional layer: Size of a feature map



- Assume the input is 7x7 and the filter is 3x3, what is the size of the output?

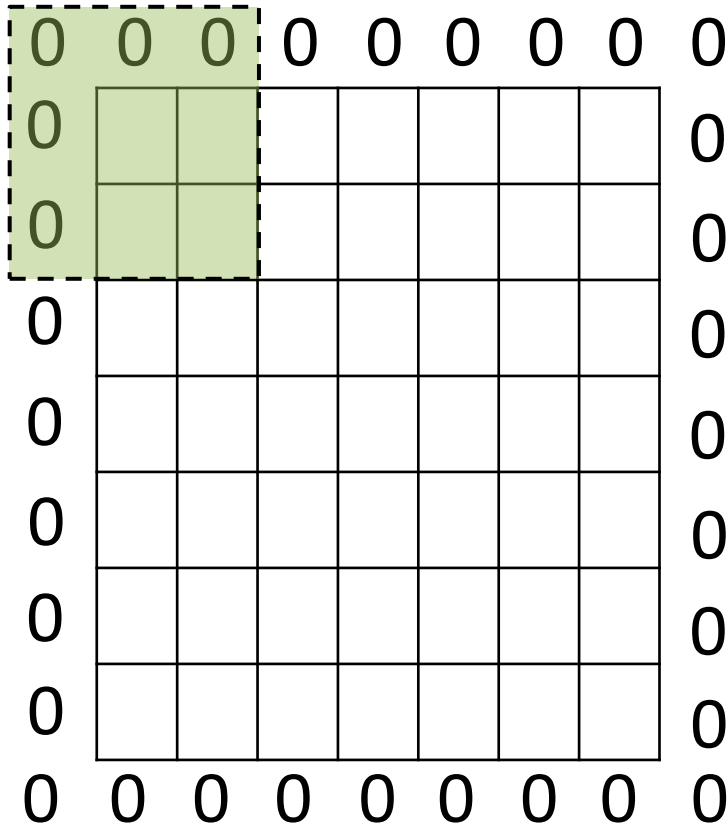
Convolutional layer: Size of a feature map



- Assume the input is 7x7 and the filter is 3x3, what is the size of the output?

5x5

Padding

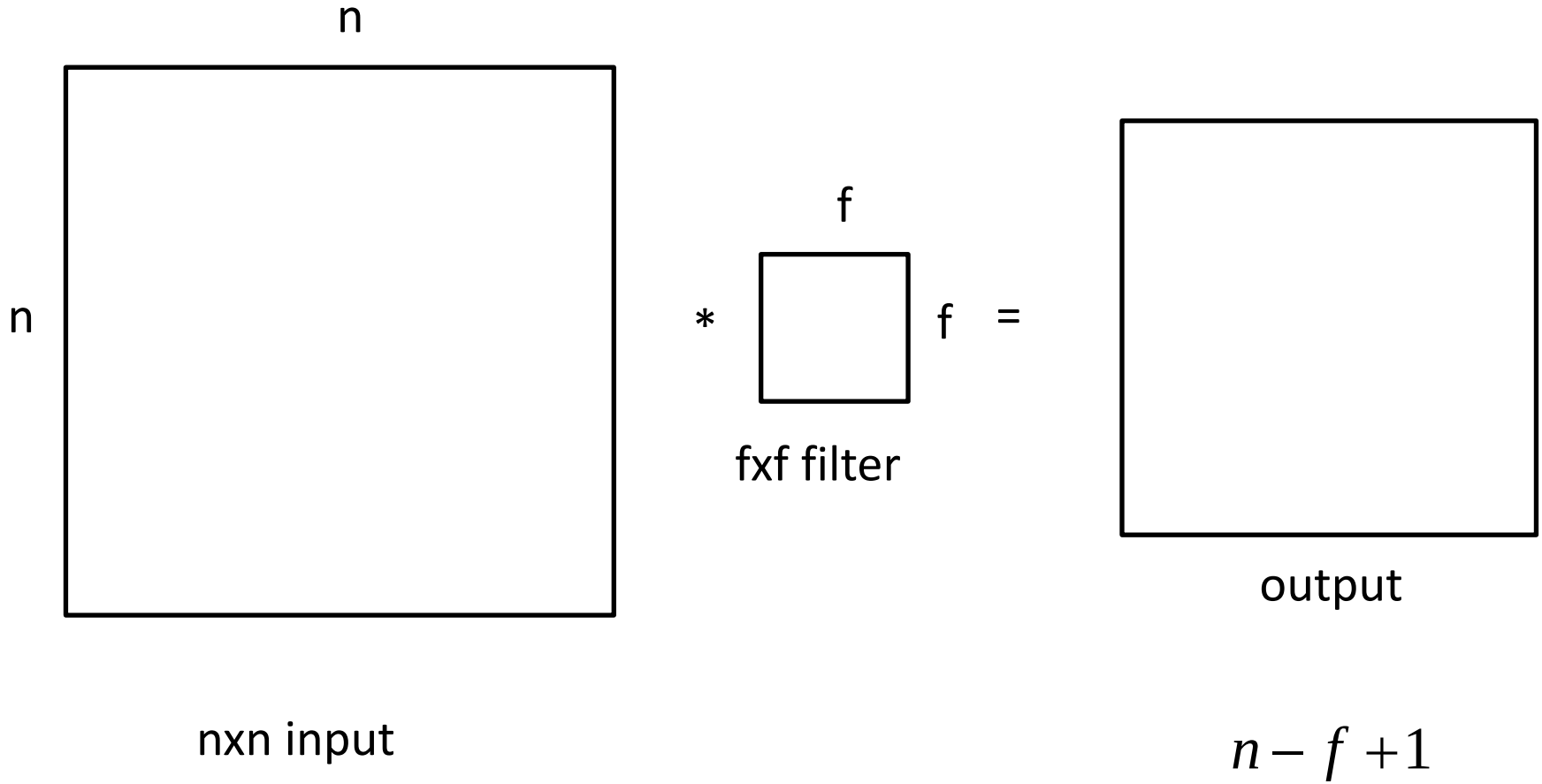


- The map will be smaller and smaller and the information on the border is lost.
- What if we want to keep the size the same?

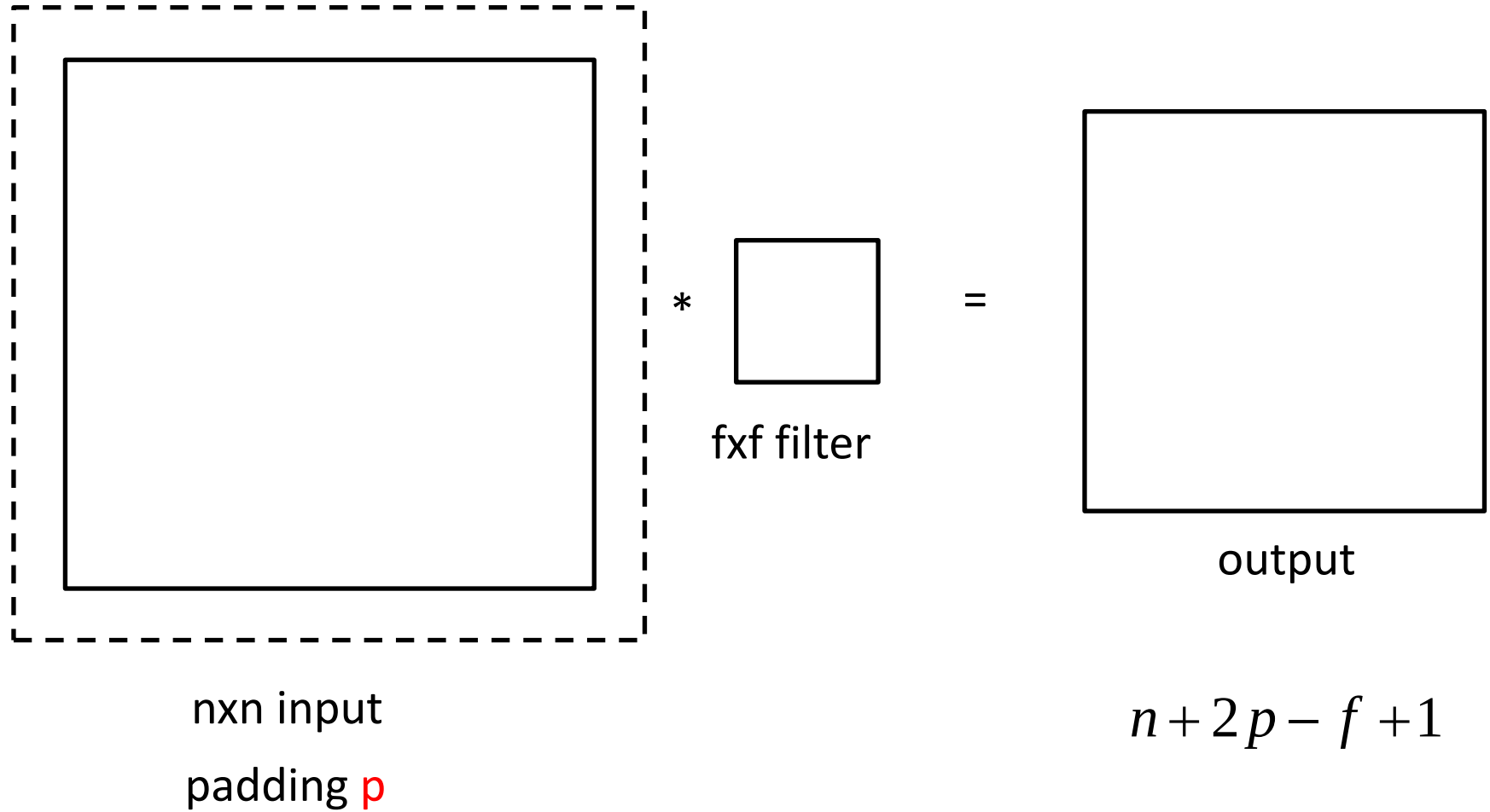
zero-padding



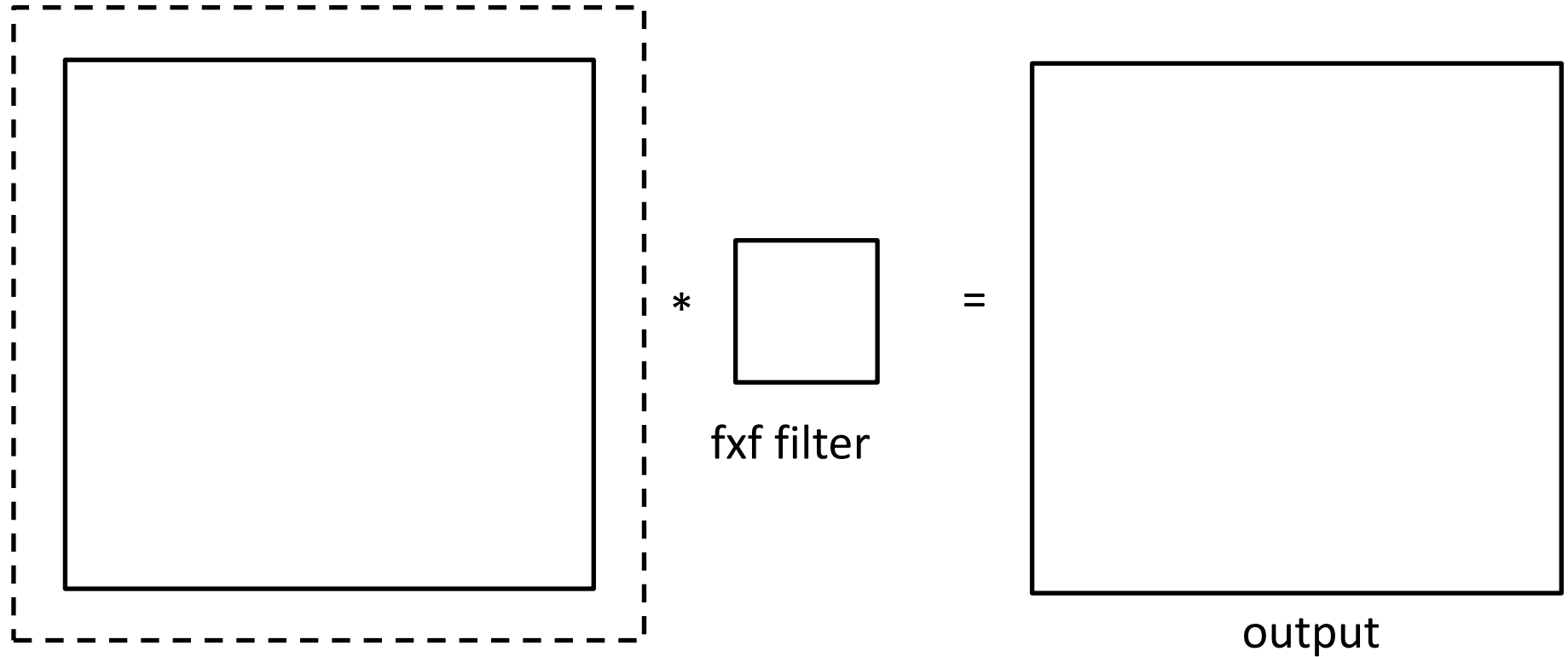
Size of the map



Size of the map



Size of the map



$n \times n$ input
padding p

$f \times f$ filter

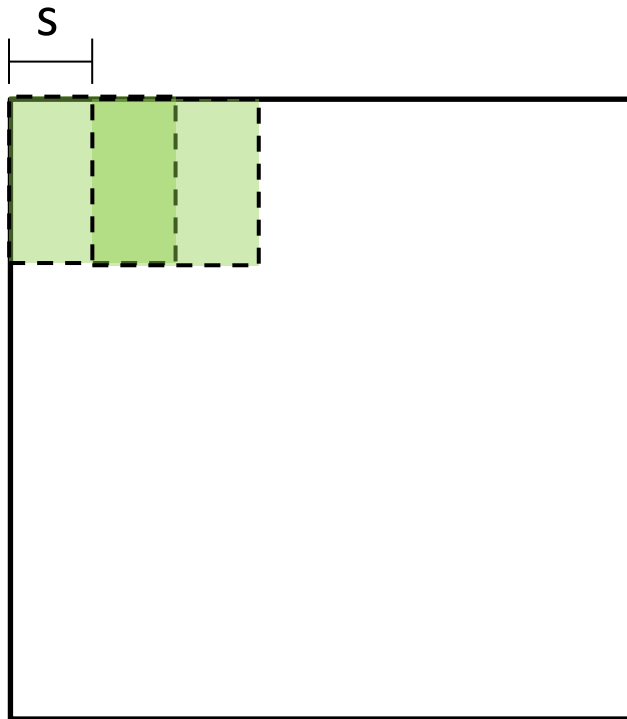
output

Keep the same size: $n + 2p - f + 1 = n$

$$p = \frac{f - 1}{2}$$

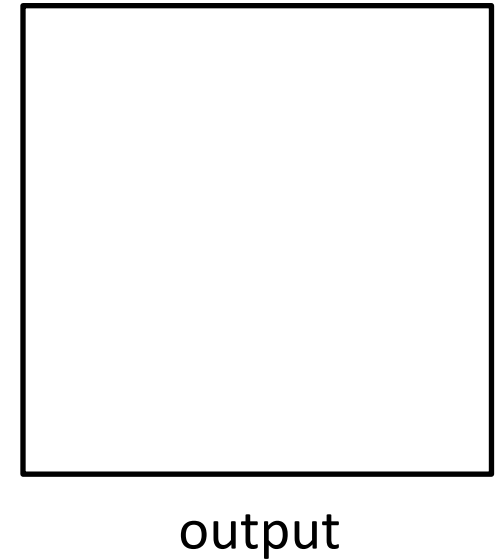


Size of the map

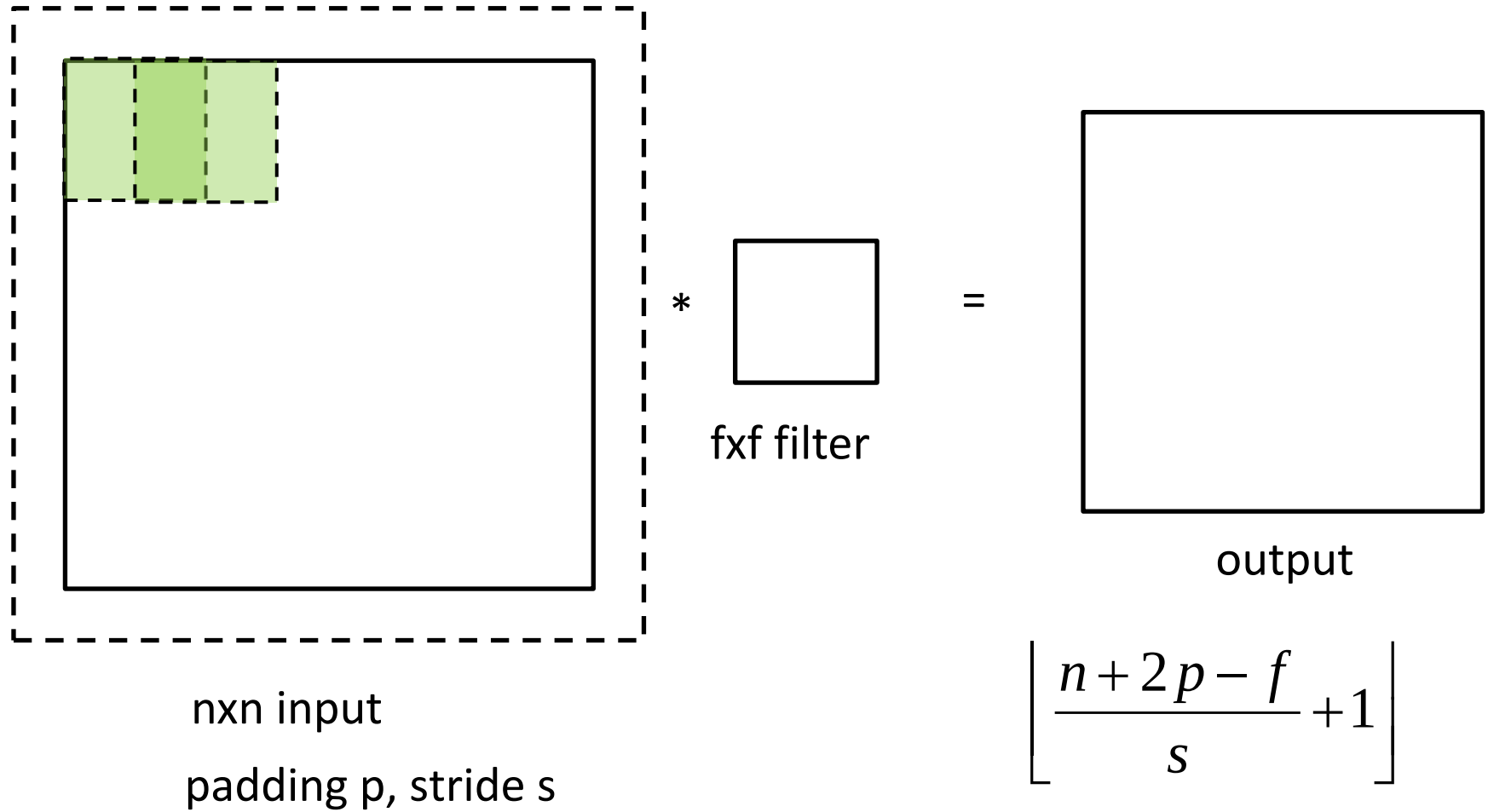


$n \times n$ input
stride s

$$* \begin{array}{c} \square \\ \text{fxf filter} \end{array} =$$



Size of the map

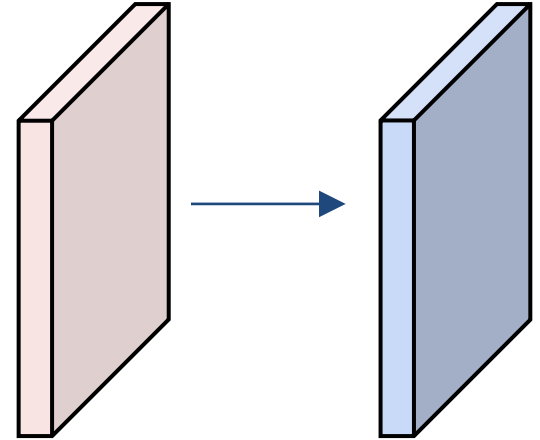


An example

Input volume: **32x32x3**

10 5x5 filters with stride 1, pad 2

Output volume size: ?



An example

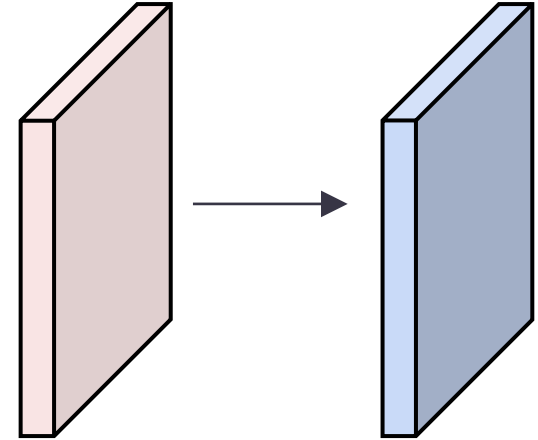
Input volume: **32x32x3**

10 **5x5** filters with stride **1**, pad **2**

Output volume size:

$(32+2*2-5)/1+1 = 32$ spatially, so

32x32x10



An example

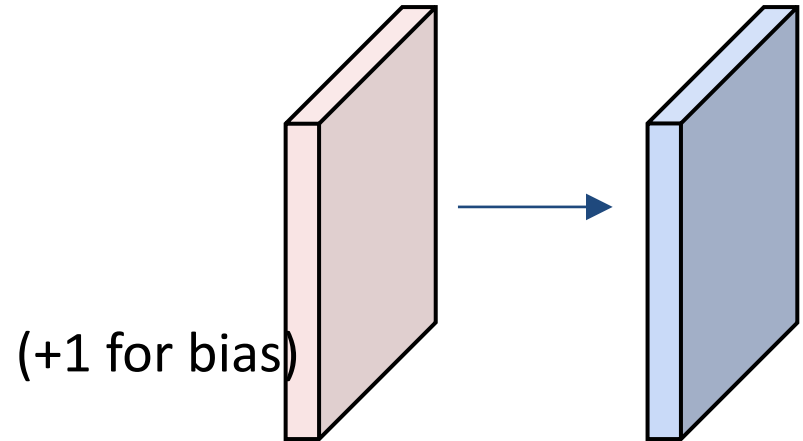
Input volume: **32x32x3**

10 **5x5** filters with stride 1, pad 2

Number of parameters in this layer?

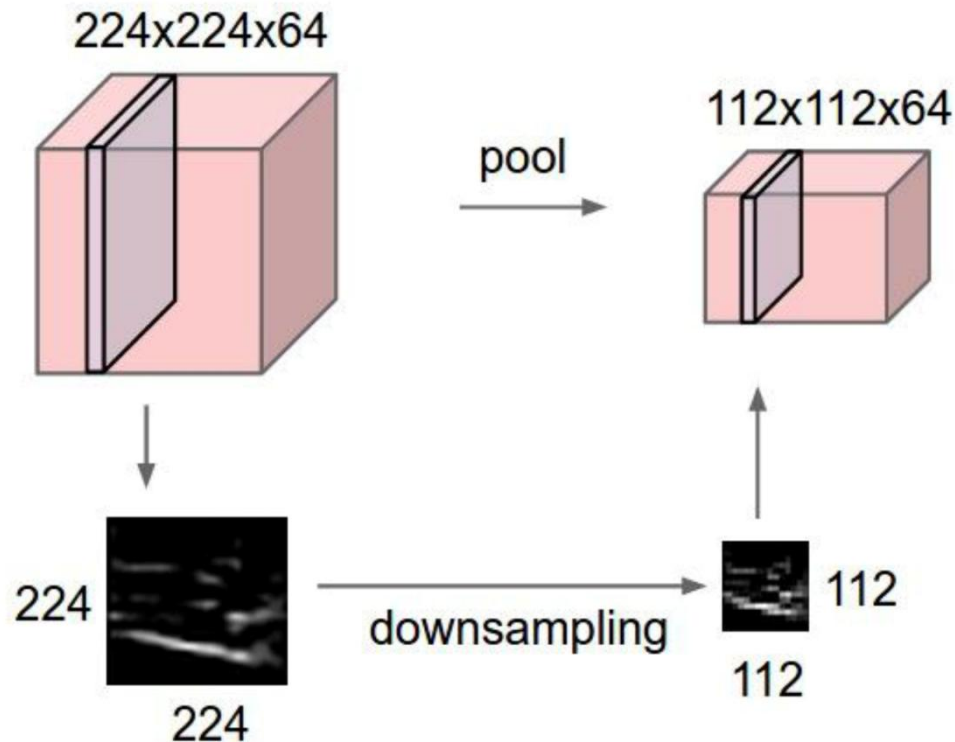
each filter has $5*5*3 + 1 = 76$ params (+1 for bias)

=> $76*10 = 760$



Pooling layer

- Makes the representations smaller and more manageable
- Operates over each feature map independently



Max pooling

- There is no parameter in the layer

Single depth slice

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

max pool with 2x2
windows and stride 2

Max pooling

- There is no parameter in the layer

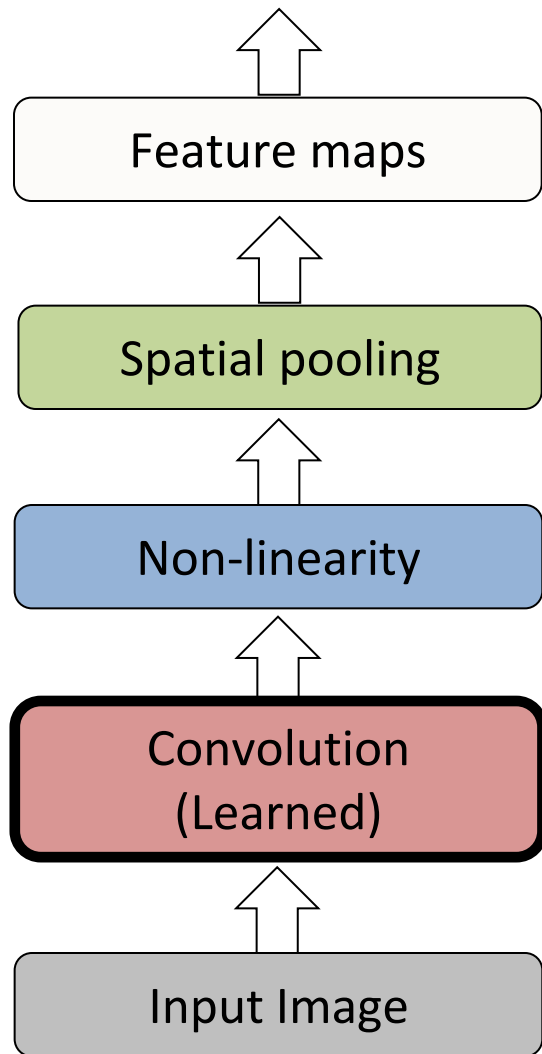
Single depth slice

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

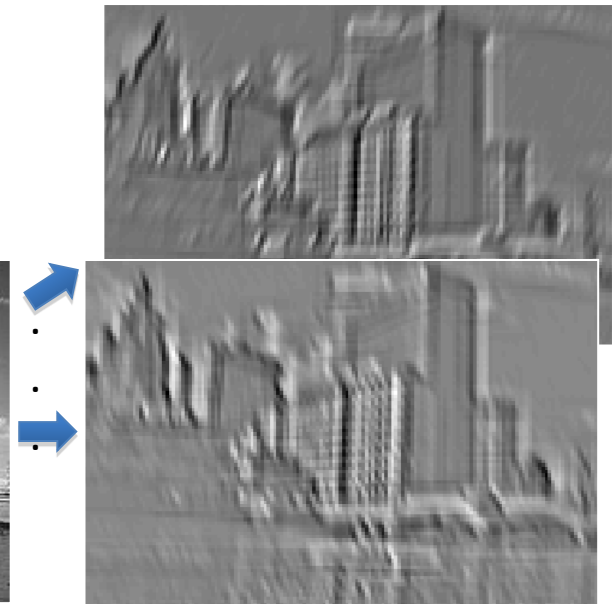
max pool with 2x2
windows and stride
2

6	8
3	4

Operations of CNN

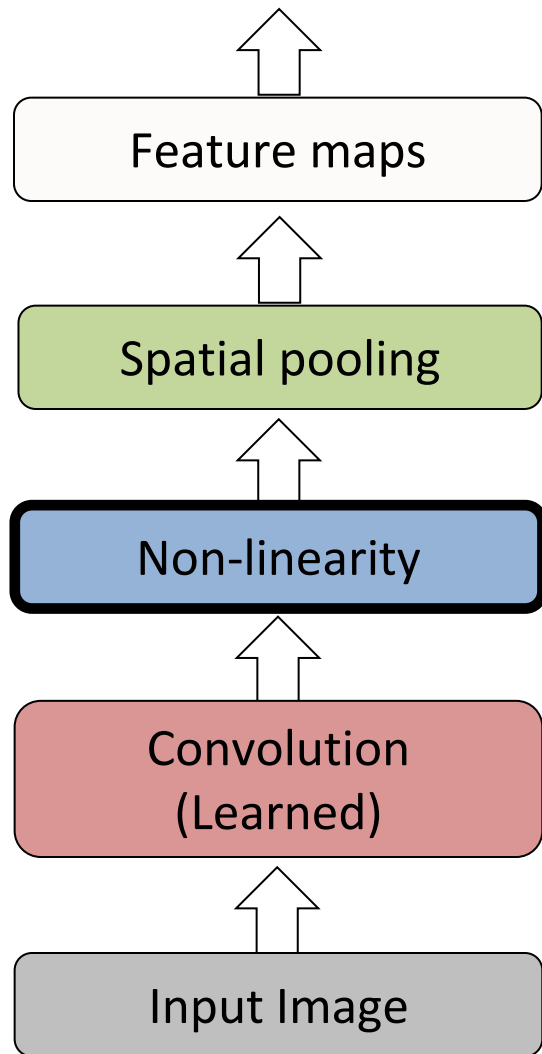


Input

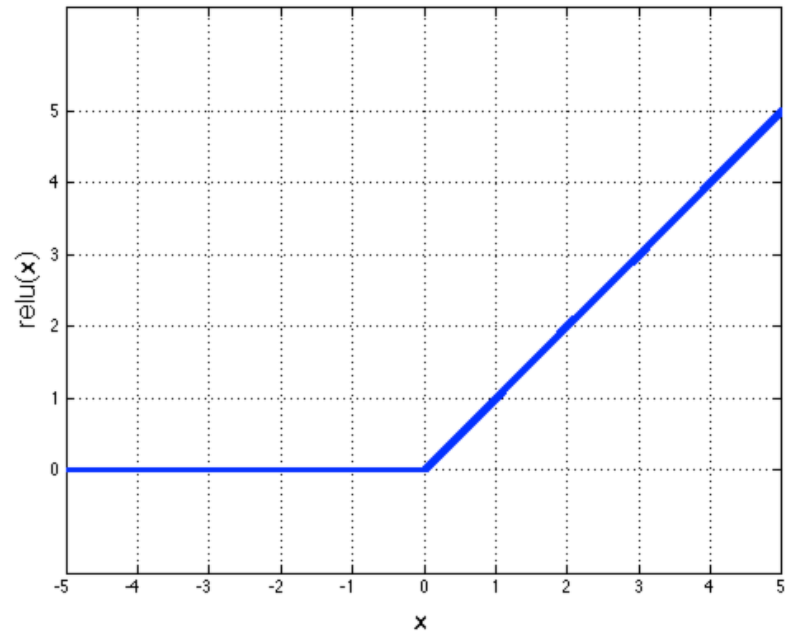


Feature Map

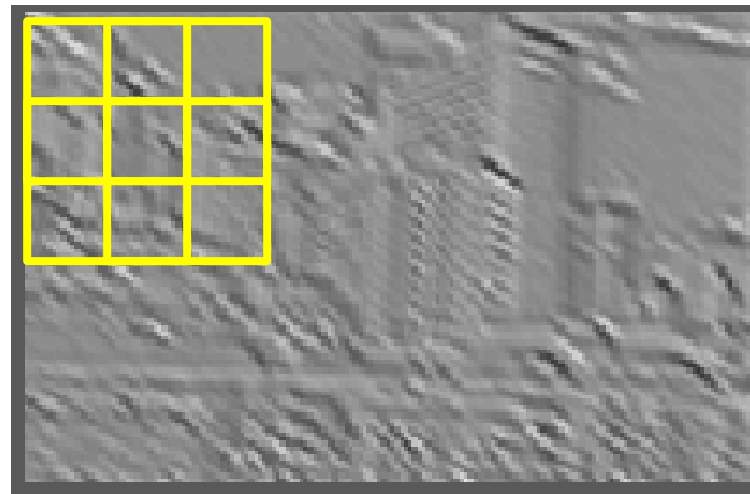
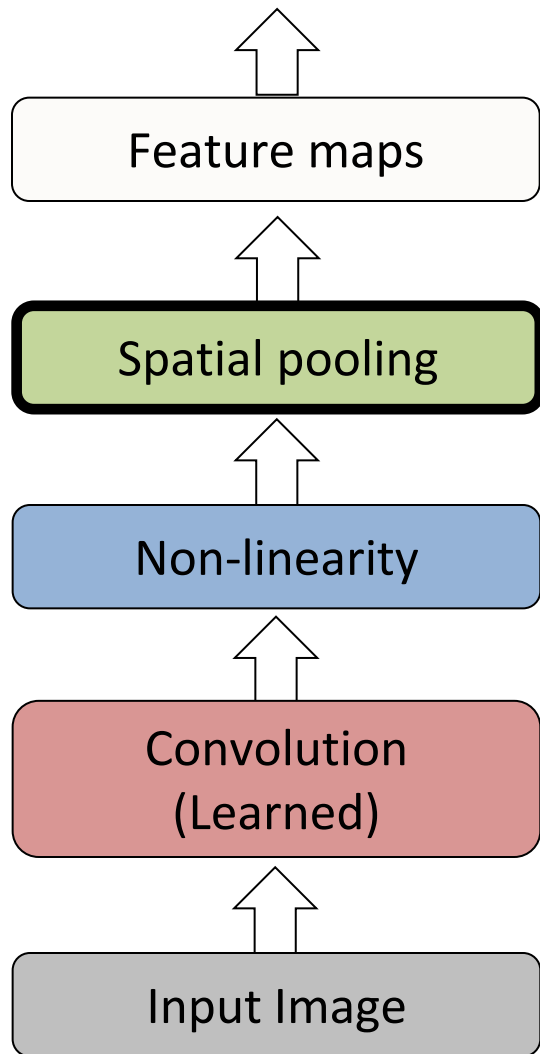
Operations of CNN



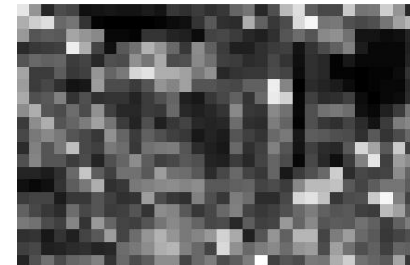
Rectified Linear Unit (ReLU)



Operations of CNN



Max



CNN vs. DNN

- Local connectivity
- Weight sharing
- Handling multiple input channels
- Handling multiple output maps

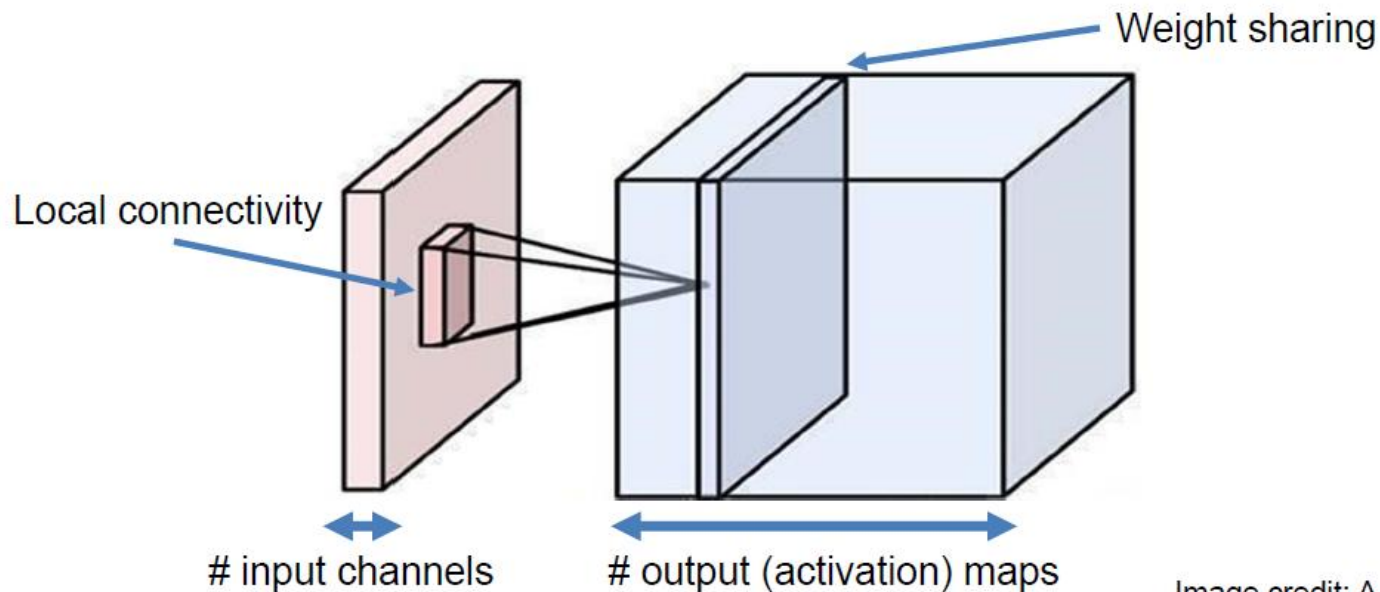
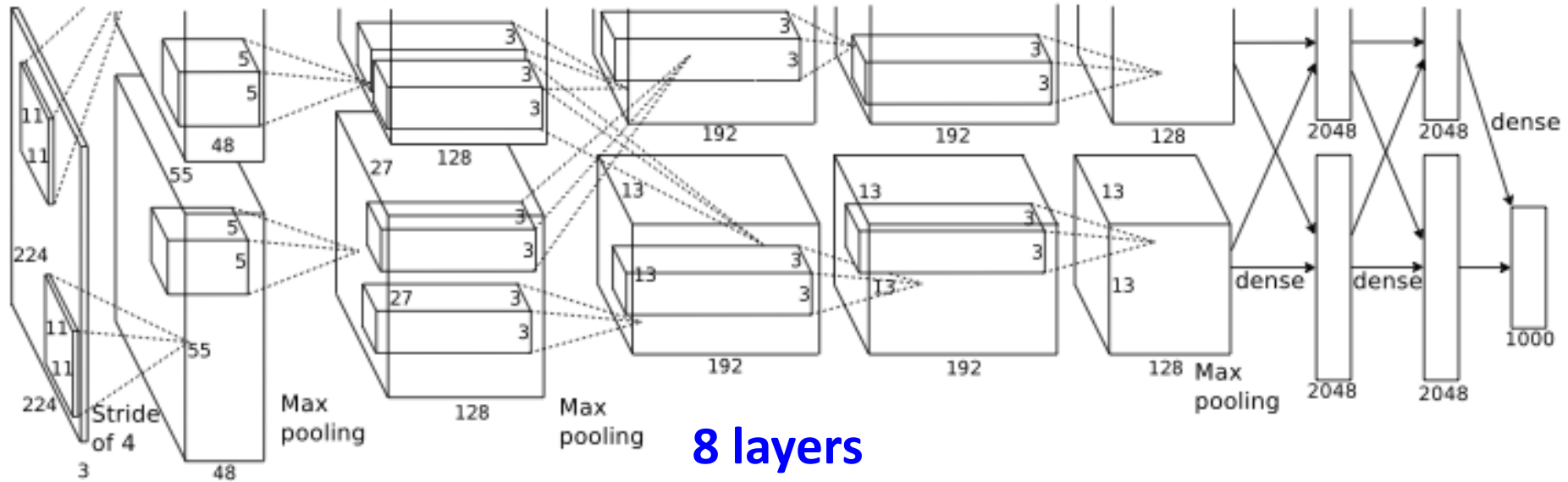


Image credit: A. Karpathy

Representative CNN: AlexNet



Input: $224 \times 224 \times 3 = 150K$

Neurons: $290400 + 186624 + 64896 + 64896 + 43264 + 4096 + 4096 + 1000 = 650K$

Weights: $11 \times 11 \times 3 \times 48 \times 2 (35K) + 5 \times 5 \times 48 \times 128 \times 2 (307K) + 128 \times 3 \times 3 \times 192 \times 4 (884K) + 192 \times 3 \times 3 \times 192 \times 2 (663K) + 192 \times 3 \times 3 \times 128 \times 2 (442K) + 6 \times 6 \times 128 \times 2048 \times 4 (38M) + 4096 \times 4096 (17M) + 4096 \times 1000 (4M) = 60M$

- More data (1.2M)
- Trained on two GPUs for a week



ImageNet dataset



IMAGENET www.image-net.org

22K categories and **15M** images

- Animals
 - Bird
 - Fish
 - Mammal
 - Invertebrate
- Plants
 - Tree
 - Flower
 - Food
 - Materials
- Structures
 - Artifact
 - Tools
 - Appliances
 - Structures
- Person
 - Scenes
 - Indoor
 - Geological Formations
 - Sport Activities



Deng, Dong, Socher, Li, Li, & Fei-Fei, 2009

ImageNet large scale visual recognition challenge (ILSVRC)

The Image Classification Challenge:

1,000 object classes

1,431,167 images



Output:

Scale

T-shirt

Steel drum

Drumstick

Mud turtle



Output:

Scale

T-shirt

Giant panda

Drumstick

Mud turtle



Russakovsky et al. IJCV 2015



ImageNet ISLVRC 2012-2014: Object Recognition

Best non-convnet in 2012: 26.2%

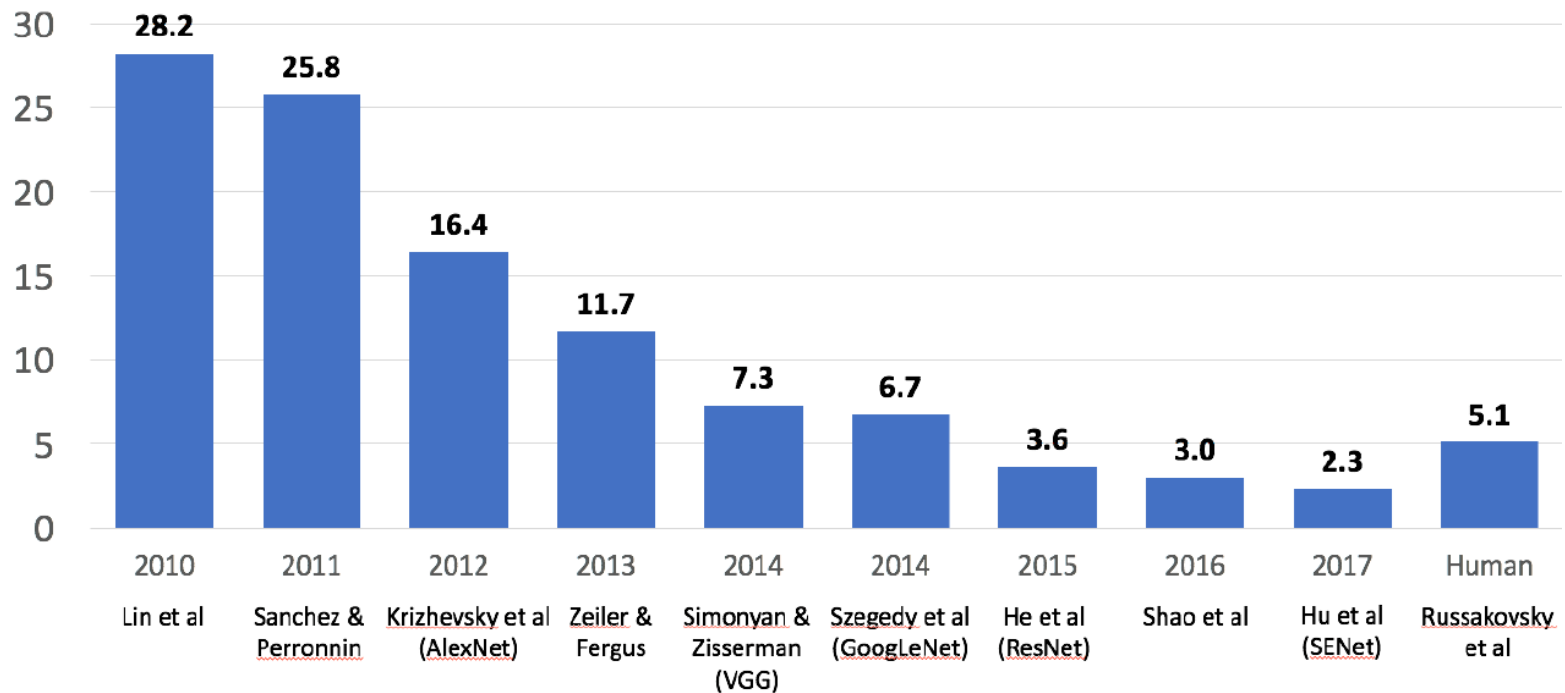
Team	Year	Place	Error (top-5)	External data
SuperVision – Toronto (7 layers)	2012	-	16.4%	no
SuperVision	2012	1st	15.3%	ImageNet 22k
Clarifai – NYU (7 layers)	2013	-	11.7%	no
Clarifai	2013	1st	11.2%	ImageNet 22k
VGG – Oxford (16 layers)	2014	2nd	7.32%	no
GoogLeNet (19 layers)	2014	1st	6.67%	no
<u>Human expert</u> *			5.1%	

Team	Method	Error (top-5)
DeepImage - Baidu	Data augmentation + multi GPU	5.33%
PReLU-nets - MSRA	Parametric ReLU + smart initialization	4.94%
BN-Inception ensemble - Google	Reducing internal covariate shift	4.82%



ILSVRC winning algorithms from 2010 ~ 2017

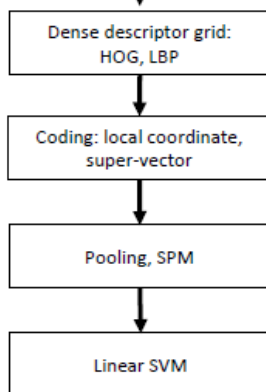
- Since 2012, **convolutional neural networks (CNN)** has become the most important tool for object recognition



ILSVRC winning algorithms from 2010 ~ 2015

Year 2010

NEC-UIUC

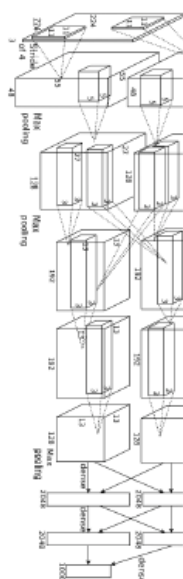


[Lin CVPR 2011]

[Lion image](#) by Swissfrog is licensed under [CC BY 3.0](#)

Year 2012

SuperVision

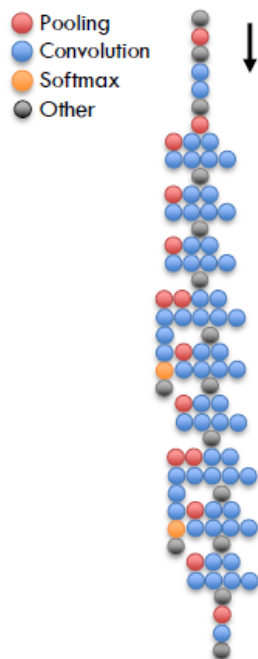


[Krizhevsky NIPS 2012]

Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

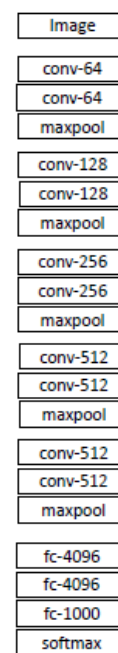
Year 2014

GoogLeNet



[Szegedy arxiv 2014]

VGG



[Simonyan arxiv 2014]

Year 2015

MSRA



[He ICCV 2015]



Reference

- Chapter 9, Convolutional Networks, in the deep learning textbook



Thank You for Your Attention!

THANK YOU FOR YOUR ATTENTION!

Yen-Yu Lin (林彥宇)

Email: lin@cs.nycu.edu.tw

URL: <https://www.cs.nycu.edu.tw/members/detail/lin>

